



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

Gra planszowa 2.5D z uczeniem maszynowym komputerowych graczy

Jagoda WIĘCEK

Nr albumu: 300503

Kierunek: Informatyka

Specjalność: Grafika Komputerowa i Oprogramowanie

PROWADZĄCY PRACĘ

Dr inż. Ewa Lach

KATEDRA Grafiki, Wizji Komputerowej i Systemów Cyfrowych

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2025

Tytuł pracy

Gra planszowa 2.5D z uczeniem maszynowym komputerowych graczy

Streszczenie

Praca przedstawia cyfrową implementację gry planszowej Rummikub w środowisku Unity. Stworzona gra w przestrzeni 2.5D, umożliwiającą rywalizację użytkownika z komputerowym graczem uczącym się. Zaimplementowano mechaniki zgodne z zasadami gry, takie jak tworzenie sekwencji, obsługa jokerów i pierwsza tura wymagająca minimum 30 punktów. Do implementacji gracza wykorzystano **ML-Agents**, stosując algorytm uczenia ze wzmacnianiem, gdzie agent wykonuje akcje, otrzymując nagrody lub kary.

Słowa kluczowe

Model, agent, unity, uczenie maszynowe

Thesis title

2.5D board game with machine learning for computer players

Abstract

The work presents a digital implementation of the board game Rummikub in the Unity environment. The game, created in a 2.5D dimension, allows the user to compete against a computer learning player. Mechanics in line with the rules of the game were implemented, such as sequence creation, joker handling and a first turn requiring a minimum of 30 points. ML-Agents were used to implement the player, using a reinforcement learning algorithm where the agent performs actions and receives rewards or penalties.

Key words

Model, agent, unity, machine learning

Spis treści

1	Wstęp	1
1.1	Cel pracy	1
1.2	Zakres pracy	1
1.3	Charakterystyka rozdziałów	2
2	Analiza tematu	3
2.1	Sformułowanie problemu	3
2.2	Znane rozwiązania	4
3	Wymagania i narzędzia	7
3.1	Wymagania funkcjonalne	7
3.2	Wymagania нефункционалне	7
3.3	Opis narzędzi	8
3.3.1	Unity	8
3.3.2	C#	9
3.3.3	Python	9
3.3.4	ML-Agents Toolkit	9
3.4	Przypadki użycia	10
4	Specyfikacja zewnętrzna	11
4.1	Wymagania sprzętowe i programowe	11
4.2	Uruchomienie	11
4.2.1	Tryb nauki gracza AI	11
4.3	Sposób obsługi	13
4.3.1	Zasady gry	14
4.3.2	Ekran aplikacji	15
4.3.3	Interakcja z płytkami gry	23
4.3.4	Predefiniowani gracze komputerowi	25
4.3.5	Nauka gracza komputerowego	25

5	Specyfikacja wewnętrzna	29
5.1	Struktura projektu	29
5.2	Moduł planszy gry	30
5.3	Moduł interfejsu użytkownika	33
5.4	Moduł kontrolera	34
5.5	Moduł logiki gry	35
5.6	Moduł użytkownika	36
5.7	Moduł gracza komputerowego	36
5.8	Moduł uczenia maszynowego	37
5.9	Struktury danych	39
5.10	System przemieszczania	39
5.11	Biblioteka ML-Agents	40
5.12	Klasa PlayerIA	43
6	Weryfikacja i walidacja	45
6.1	Testowanie	45
6.1.1	Testy manualne	45
6.1.2	Modele	47
6.2	Wykryte błędy	49
7	Podsumowanie i wnioski	51
7.1	Napotkane problemy	51
7.2	Możliwy rozwój	52
	Bibliografia	54
	Spis skrótów i symboli	57
	Źródła	59
	Lista dodatkowych plików, uzupełniających tekst pracy	61
	Spis rysunków	64
	Spis tabel	65

Rozdział 1

Wstęp

Gry planszowe są formą rozrywki dla ludzi od wieków jak i formą intelektualnego wyzwania. We współczesnych czasach znaczna część życia podlega cyfryzacji, od komunikacji do płatności bankowych. Z tego powodu również wszelkie gry planszowe są przenoszone do cyfrowego świata. Takie rozwiązanie pozwala na rozpoczęcie rozgrywki ulubionej gry planszowej w świecie online wraz z graczami z całego świata bez żadnych ograniczeń. Nie zawsze istnieje możliwość znalezienia współgraczy, a ograniczenia techniczne mogą dodatkowo to utrudniać. Komputerowi gracze, szczególnie ci oparte na modelach uczących się, mogą stanowić alternatywę, zapewniając większą rozrywkę.

1.1 Cel pracy

Celem pracy jest zaimplementowanie gry planszowej 2.5D w unity. Dodatkowo w ramach gry zostanie zaimplementowany gracz komputerowy z wykorzystaniem uczenia maszynowego. Gra będzie przeznaczona wyłącznie do rozgrywki jednoosobowej, w której gracz zmierzy się z zaimplementowanymi graczami komputerowymi.

1.2 Zakres pracy

- Przegląd algorytmów uczenia maszynowego dostępnego w unity.
- Zaprojektowanie mapy do gry.
- Zaimplementowanie gry planszowej Rummikub 2.5D w unity.
- Zaimplementowanie graczy komputerowych z algorytmem uczenia maszynowego oraz graczy komputerowych z predefiniowanymi ruchami.

1.3 Charakterystyka rozdziałów

- **Wstęp** - Przedstawienie celu oraz zakres pracy, określenie grup docelowych
- **Analiza tematu** - Zawiera ogólną analizę wszystkich możliwych rozwiązań tego projektu.
- **Wymagania i narzędzia** - Przedstawienie narzędzi, wykorzystanych przy tworzeniu projektu, wymagania funkcjonalne, wymagania нефunkcjonalne oraz diagram UML przedstawiający przypadki użycia.
- **Specyfikacja zewnętrzna** - Przetawia minimalne wymagania sprzętowe, jakie użytkownik powinien spełniać, aby korzystać z aplikacji. Zawiera również instrukcje dotyczące instalacji, aktywacji, zasady gry oraz ogólny opis użytkowania aplikacji.
- **Specyfikacja wewnętrzna** - Przedstawia ogólny opis struktury projektu, najważniejsze klasy w projekcie oraz opis działania i implementacji modelu uczenia maszynowego.
- **Weryfikacja i walidacja** - Rozdział przedstawia proces testowania aplikacji oraz jakie błędy zostały wykryte na przedziale całej implementacji projektu.
- **Podsumowanie i wnioski** - Przedstawia przemyślenia o projekcie, napotkane błędy oraz możliwy rozwój aplikacji w przyszłości.

Rozdział 2

Analiza tematu

Tematem pracy jest cyfrowa gra planszowa z wykorzystaniem algorytmów sztucznej inteligencji. Kluczowym problemem jest zaimplementowanie inteligentnego gracza, który poprzez system prób i błędów, nauczy się grać. Do realizacji tego projektu została wybrana gra planszowa o nazwie Rummikub.

Rummikub to gra planszowa, w której gracze rywalizują, układając płytki w:

- grupy - te same liczby w różnych kolorach
- serie - kolejne liczby w tym samym kolorze.

Celem graczy jest jako pierwszy wyłożyć wszystkie płytki, przy czym pierwszy ruch wymaga uzyskania układów o sumie co najmniej 30 punktów. Gra kończy się, gdy jeden z graczy wyłoży wszystkie płytki. Wynik pozostałych graczy zależy od płytek, jakie pozostały na ich stojakach. Każdy gracz powinien dążyć do uzyskania jak najmniejszego wyniku.

2.1 Sformułowanie problemu

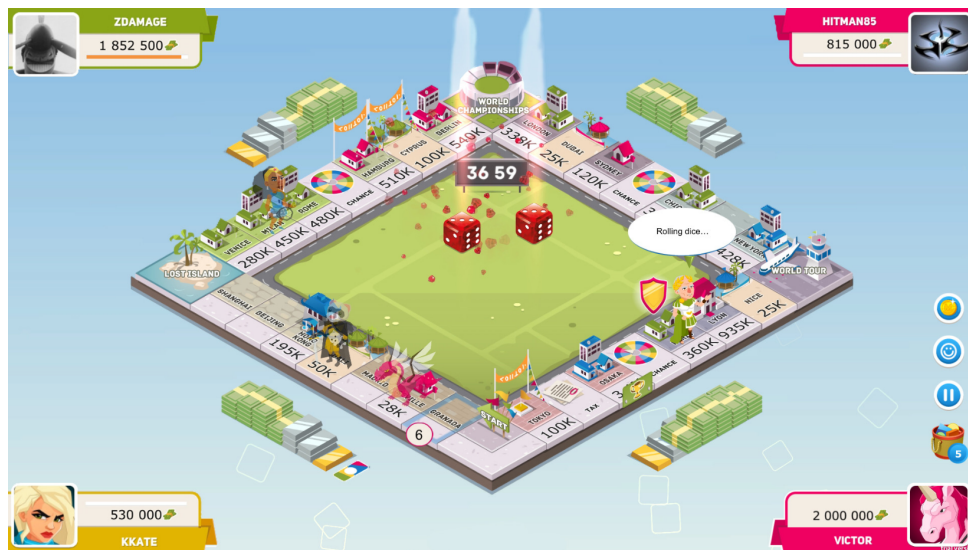
Cyfrowe wersje gier planszowych często są projektowane z myślą o rozgrywkach online z innymi graczami za pośrednictwem internetu. Jednak implementacja komputerowego przeciwnika opartego na algorytmach uczenia maszynowego wymaga innego podejścia, skoncentrowanego na samodzielnym procesie uczenia się przez model. Algorytmy AI potrzebują dużej liczby powtórzeń, aby efektywnie nauczyć się zasad gry, dlatego proces uczenia zostanie przeprowadzony w środowisku symulacyjnym, bez udziału żywych graczy.

W projekcie uwzględniono jednego żywego gracza, który będzie mógł rywalizować z komputerowymi przeciwnikami. W grze znajdują się zarówno gracze komputerowi wykonujący predefiniowane ruchy, jak i gracz sterowany algorytmem uczenia maszynowego, który będzie podejmował decyzje na podstawie wytrenowanego modelu.

2.2 Znane rozwiązania

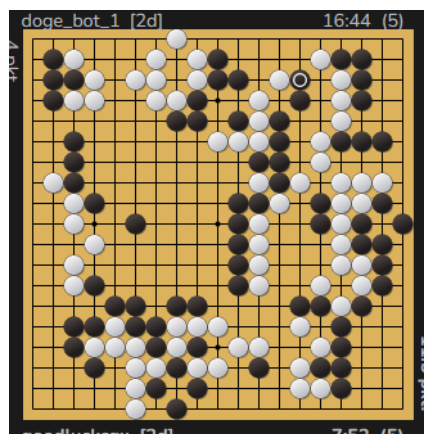
Na rynku jest wiele gier planszowych, najpopularniejsze z nich doczekały się swojej cyfrowej wersji. Są nimi między innymi:

- Monopoly - Jedna z najpopularniejszych gier planszowych. Gra oparta na finansach i rywalizacji z innymi graczami została wydana w wersji cyfrowej, również są dostępne online, jedną z nich można znaleźć na stronie <https://www.kurnik.pl/netopol/>. Gra doczekała się również gier o charakterze monopoly, takich jak Business Tour, którą można znaleźć i zainstalować na stronie https://store.steampowered.com/app/397900/Business_Tour__Board_Game_with_Online_Multiplayer/



Rysunek 2.1: Zrzut ekranu gry Business Tour

- Go - Chińska gra planszowa, popularna również w Korei i Japonii. Celem gry jest zajęcie większej powierzchni planszy od przeciwnika. Gra go ma swoją wersję cyfrową na stronach internetowych, między innymi na stronie <https://online-go.com>. Na rysunku 2.2 jest pokazana przykładowa gra na podanej stronie



Rysunek 2.2: Zrzut ekranu gry Go na stronie online-go.com

- Catan - Strategiczna gra planszowa. Gracze mają za zadanie zyskiwać punkty zwycięstwa poprzez wykonywanie konkretnych ruchów na mapie. W grze stawia się budynki i drogi w miejscach strategicznych, co daje graczowi przewagę nad innymi. Wygrywa gracz, który uzyska ustaloną ilość punktów zwycięstwa. Gra planszowa **Catan** ma swoją wersję cyfrową dostępną dla graczy z całego świata, jedną z nich można znaleźć na stronie https://store.steampowered.com/app/544730/Catan_Universe/?l=polish, na rysunku 2.3 widać, jak wygląda interfejs gry.



Rysunek 2.3: Zrzut ekranu gry Catan

- Szachy - Jedna z najbardziej popularnych strategicznych gier planszowych. Gra, oparta na rywalizacji z drugim graczem, posiada wiele wersji cyfrowych, jedna wersja jest dostępna na stronie <https://www.chess.com>, wygląd gry można zobaczyć na rysunku 2.4. Pewne wersje szachów w wersji online posiada zaimplementowane algorytmy sztucznej inteligencji, jedną z nich jest silnik gry Stockfish - Open Source Chess Engine [1], gdzie znajduje się pełna dokumentacja o zastosowanych algorytmach.



Rysunek 2.4: Zrzut ekranu rozgrywki szachów ze strony chess.com

- Rummikub - Wydana gra online Rummikub jest nastawiona na grę sieciową do komunikacji innych graczy z całego świata, w grę online od oficjalnego producenta można zagrać za darmo na stronie <https://rummikub.com/play-online/>. Gra jest bez algorytmów sztucznej inteligencji, zaimplementowana w grafice 2D, co można zauważyć na rysunku 2.5.



Rysunek 2.5: Zrzut ekranu gry Rummikub online

Rozdział 3

Wymagania i narzędzia

W tym rozdziale przedstawiono wymagania funkcjonalne i нефункционалне projektu oraz narzędzia technologiczne i programistyczne, które zostały wykorzystane do jego realizacji. Opisane zostały kluczowe zasoby niezbędne do wdrożenia rozwiązania, takie jak środowiska programistyczne, biblioteki oraz wymagany sprzęt.

3.1 Wymagania funkcjonalne

Sformułowano następujące wymagania funkcjonalne aplikacji

- Wizualizacja rozgrywki.
- Interakcja użytkownika z elementami gry.
- Zapobieganie zachowaniom graczy wbrew zasadom gry.
- Interakcja użytkownika z grą za pośrednictwem klawiatury i myszy.
- Zaimplementowanie gracza komputerowego z predefiniowanymi zachowaniami zgodnie z zasadami gry.
- Implementacja różnych trybów gry.
- Zaimplementowanie gracza komputerowego z algorytmem uczenia maszynowego.

Zdefiniowano następujące wymagania funkcjonalne aplikacji

3.2 Wymagania нефункционалне

- Kompatybilność :
 - Gra musi działać na systemach Windows.

- **Użyteczność :**

- Interfejs użytkownika powinien być intuicyjny i zrozumiały dla nowych graczy.
- Kolorystyka i oznaczenia płytek powinny odpowiadać klasycznym zasadom gry Rummikub.

3.3 Opis narzędzi

W tej części przedstawiono narzędzia, które zostały wykorzystane do zrealizowania projektu.

3.3.1 Unity

Środowisko gry zostało zaimplementowane w silniku Unity w wersji 2022.3.48f1. Unity jest wszechstronnym silnikiem do tworzenia gier i aplikacji 3D jak i 2D zaimplementowany przez Unity Technologies [12]. Jest szeroko stosowany nie tylko do gier wideo, ale również do symulatorów, wizualizacji interaktywnych oraz aplikacji mobilnych. Unity oferuje dużo możliwości, takie jak:

- Tworzenie modeli 2D i 3D.
- Animacje i symulacja fizyki.
- Renderowanie grafiki.
- Wykorzystanie narzędzi, takich jak okulary wirtualnej rzeczywistości oraz rozszerzonej rzeczywistości.
- Tworzenie gier z trybem wieloosobowym.

Projekt w Unity składa się z:

- Scen - to podstawowa jednostka organizacyjna w Unity, reprezentuje oddzielne etapy symulacji. Przykładowo w grze, będą osobne sceny na menu i rozgrywkę.
- Obiektów - są to elementy sceny, są nimi, między innymi, obiekty 3D, przyciski, światło, kamera.
- Komponentów - to moduły dodawane do obiektów, dzięki nim można przykładowo zmieniać pozycję czy rotację obiektu.
- Skryptów - to fragmenty kodu, najczęściej pisane w C#, dzięki nim można implementować unikalne zachowania obiektów.
- Prefabrykatów - to wzorzec obiektu, który można wielokrotnie używać w scenach.

3.3.2 C#

C# (C-Sharp) to nowoczesny, obiektowy język programowania opracowany przez firmę Microsoft [4]. Łączy zalety języków C++ oraz Java, co czyni go uniwersalnym przy użytkowaniu. Może być wykorzystany do tworzenia gier wideo czy aplikacji mobilnych.

C# wyróżnia się następującymi cechami:

- Programowanie obiektowe.
- Zarządzanie pamięcią - unkcja garbage collector zapobiega wyciekom pamięci.
- Uniwersalność – umożliwia tworzenie aplikacji dla systemów Windows, macOS, Android, iOS oraz implementować zachowania obiektów w środowisku Unity.

C# w projekcie został wykorzystany do zaimplementowania logiki gry, komunikacji z modelem oraz zarządzania interakcji między graczem a środowiskiem w Unity.

3.3.3 Python

Python to obiektowy, wysokopoziomowy język programowania, głównie znany z prostoty i przejrzystości kodu. Więcej informacji na temat języka Python znajduje się w dokumentacji Python Software Foundation [3]. Najważniejsze cechy języka Python to:

- Bogata biblioteka standardowa – zawiera moduły do pracy z danymi, plikami, sieciami, obliczeniami naukowymi i innymi zadaniami.
- Przenośność i uniwersalność – Python działa na większości systemów operacyjnych, takich jak Windows, macOS czy Linux i jest wykorzystywany w aplikacjach desktopowych oraz webowych.
- Poprzez istnienie wielu zewnętrznych bibliotek, takich jak NumPy, TensorFlow, PyTorch, idealnie nadaje się do analizy danych oraz wspomaganie tworzenia podsumowań wyników badań naukowych.

W niniejszym projekcie Python w wersji 3.9 jest wykorzystany jako środowisko do implementacji modelu uczenia maszynowego, jest to konieczne w przypadku biblioteki ML-Agents.

3.3.4 ML-Agents Toolkit

ML-Agents (Machine Learning Agents) to ,opracowany przez Unity Technologies, zestaw narzędzi do wykorzystywania technik sztucznej inteligencji i uczenia maszynowego w projektach w środowisku Unity. Narzędzie pozwala na implementację agentów, którzy

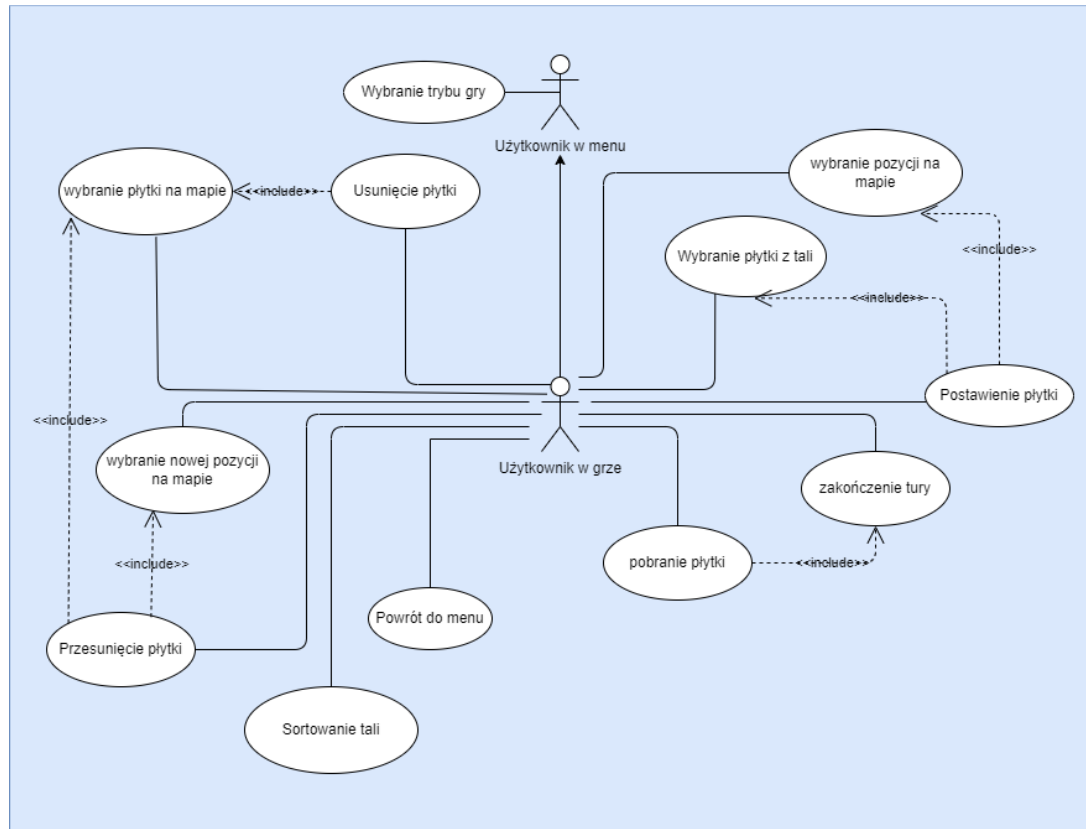
mogą podejmować decyzje w zaimplementowanym środowisku. Więcej informacji na temat ML-Agents Toolkit można znaleźć w repozytorium GitHub Unity Technologies [9]. Kluczowymi cechami ML-Agents:

- Uczenie ze wzmacnianiem - agenci uczą się na bazie dostawanych kar i nagród, jakie otrzymują.
- Narzędzie jest uniwersalne, można je zaimplementować do prostych jak i zaawansowanych gier, w których zaimplementowani agenci muszą podejmować decyzje.
- Agentów można zaimplementować do różnego rodzaju scenariuszy.

W tym projekcie, narzędzie, do poprawnej nauki wykorzystuje środowisko Python oraz Unity. Unity jest miejscem implementacji akcji, obserwacji, nagród i kar dla modelu. Środowisko Python jest wykorzystywane do implementacji modelu oraz rozpoczynania nauki.

3.4 Przypadki użycia

Rysunek 3.1 przedstawia diagram przypadków użycia prezentując możliwe funkcje dla gracza w menu oraz w grze.



Rysunek 3.1: Diagram przypadków użycia

Rozdział 4

Specyfikacja zewnętrzna

4.1 Wymagania sprzętowe i programowe

Minimalne wymagania zostały opisane w dokumentacji Unity [13], użytkownik powinien posiadać komputer z tymi bądź lepszymi komponentami:

- Windows 10 and Windows 11 tylko w wersji 64-bitowej
- CPU x64 z obsługą zestawu instrukcji SSE2.
- GPU kompatybilne z DX10, DX11 i DX12.

System był testowany na komputerze z komponentami:

- AMD Ryzen 9 7940HS w/ Radeon 780M Graphics 4.00 GHz
- 16,0 GB pamięci RAM
- System 64-bitowy Windows 11
- NVIDIA GeForce RTX 4050

4.2 Uruchomienie

Poniżej zostaną przedstawione biblioteki, których instalacja jest wymagana do uruchomienia aplikacji.

4.2.1 Tryb nauki gracza AI

- Unity

Do uruchomienia projektu wymagana jest instalacja Unity w wersji co najmniej 2022.3 . Więcej informacji, na temat instalacji, znajduje się [8]. Aby zainstalować Unity, należy:

1. Pobrać Unity Hub z oficjalnej strony: <https://unity.com/download>.
2. Zainstalować Unity Hub i dodać wersję edytora Unity 2022.3 lub nowszą.

- Python

Konieczne jest zainstalowanie Pythona w wersji 3.8 bądź nowsza, więcej informacji można znaleźć w [2].

Należy pobrać instalator z odpowiednią wersją oficjalnej strony <https://www.python.org/downloads/>.

- Biblioteka ml-agents

Biblioteka ML-Agents Toolkit jest wymagana do wdrożenia komponentów sztucznej inteligencji w grze. Szczegółowa instrukcja instalacji znajduje się w [11]. Konieczne jest zainstalowanie jej w środowisku Python jak i w Unity zgodnie z następującymi krokami:

1. Przejdź do zakładki Okno (Window) w Unity.
2. Wybierz Menedżer Pakietów (Package Manager).
3. W okienko wyszukiwania wpisz *ml-agents*.
4. Zainstaluj wybraną wersję.
5. W plikach projektu stwórz środowisko wirtualne Python przy użyciu *Wiersza poleceń*.

```
python -m venv nazwa_folderu
```

bądź

```
py -m venv nazwa_folderu
```

6. Aktywuj środowisko przy użyciu.:

```
nazwa_folderu\Scripts\activate
```

7. Zainstaluj w środowisku wirtualnym potrzebne biblioteki.

```
py -m pip install --upgrade pip  
pip install torch
```

aby zainstalować konkretną wersję biblioteki pytorch, należy wpisać komendę.:

```
pip install torch==(wybrana wersja)
```

8. Ostatecznie należy wpisać komendę.:

```
pip install mlagents
```

Jeśli ta komenda wywoła błąd, należy wpisać.:

```
pip install mlagents --use-feature=2020-resolver
```

9. Aby zweryfikować, czy biblioteka została poprawnie zainstalowana, należy wpisać:

```
mlagents-learn --help
```

Jeśli biblioteka jest poprawnie zainstalowana, to polecenie wyświetli dostępne komendy w ramach biblioteki ml-agents.

- Aktywacja Aby rozpocząć naukę sztucznej inteligencji (ML-Agents), należy wykonać kolejne polecenia:
 1. Otwórz Unity Hub i wybierz projekt z listy w odpowiedniej wersji edytora Unity.
 2. Kliknij przycisk **Open**, aby załadować projekt.
 3. W edytorze Unity załaduj scenę **CP_AI_GAME**
 4. W Wierszu poleceń przejdź do plików gry
 5. Aktywuj środowisko wirtualne w Python
 6. Wpisz poniższą komendę:

```
mlagents-learn --run-id=nazwa
```

7. Naciśnij przycisk **Play** w Unity.

Po wykonaniu powyższej instrukcji, w środowisku wirtualnym oraz w unity zostanie aktywowana nauka modelu, którą użytkownik będzie mógł obserwować.

4.3 Sposób obsługi

Sposób obsługi zawiera szczegółową instrukcję, jak korzystać z aplikacji z perspektywy użytkownika.

4.3.1 Zasady gry

Projekt gry Rummikub jest oparty na fizycznej grze planszowej, której zasady zostały opisane dokładnie w [5]. Gra wymaga od dwóch do czterech graczy. Gra podstawowa zawiera 106 płytek, w którym jest osiem serii o wartościach od jeden do trzynaście w czterech kolorach, oraz dwa jokery. Jokery to płytki specjalne, które mogą zastąpić każdą płytkę na mapie. Zadaniem każdego gracza jest jak najszybciej wyłożyć na stół wszystkie płytki, które zawiera w swojej tabliczce. W grze znajdują się dwa typy sekwencji płytek

- Grupy - jest to sekwencja płytek, w skład której wchodzi co najmniej trzy płytki o tej samej liczbie ale różnych kolorach. Joker może zastąpić płytkę w takiej sekwencji, grupa nie może zawierać więcej niż 4 płytki
- Serie - jest to sekwencja co najmniej trzech płytek w tym samym kolorze o różnych wartościach w kolejności od najmniejszej do największej. W takiej sekwencji może znajdować się maksymalnie 13 płytek, rozpoczynającą płytką jest zawsze o numerze jeden, a kończącą o numerze trzynaście.

Każdy gracz zaczyna rozgrywkę z czternastoma płytkami. Na początku rozgrywki gracze muszą wyłożyć sekwencje i grupy o łącznej wartości trzydziestu punktów i więcej. Każda płytki ma swoją wagę punktową, zależną od liczby wypisanej na płytce. Wartością wagową jokera jest liczba jaką zastępuje na planszy. Dopóki gracz nie wykona pierwszego ruchu, nie może manipulować żadnymi płytkami na mapie. Gracz również nie może manipulować żadnymi sekwencjami na mapie w trakcie swojego pierwszego wyłożenia. Po zakończeniu pierwszej tury, gracz może wykładać grupy i serie oraz wykonywać akcje, takie jak:

- dokładanie płytek na końcu i początku sekwencji na mapie
- rozdzielać jedną większą sekwencje na dwie mniejsze
- podmieniać jokery na mapie z płytkami ze swojego stojaka.
- przebudowywać grupy na serie oraz serie na grupy

Gracz musi w każdej turze wyłożyć co najmniej jedną płytkę, jeśli nie może tego zrobić, musi pobrać płytkę z banku, co kończy jego turę i uniemożliwia wykonanie ruchu na mapie. W grze obowiązuje limit czasu. Jeśli gracz nie zakończy swoich manipulacji przed upływem czasu, wszystkie jego ruchy należy cofnąć, a gracz dobiera płytkę z głównego banku. Gra może zakończyć się w dwóch momentach:

1. gdy jeden gracz wyłoży wszystkie płytki ze swojego stojaka
2. jeśli w głównym banku skończą się płytki a gracz, który ma swoją turę, chce dobrać kolejną.

Wynik gry zależy od płytek w stojaku gracza. Na końcu gry są podliczane wyniki na podstawie wag wartości płytek. Wagi jokerów są równe 30. Wygrywa ten, którego wynik jest najmniejszy.

4.3.2 Ekran aplikacji

Rozgrywka rozpoczyna się widokiem menu głównego gry, zaprezentowanym na rysunku 4.1

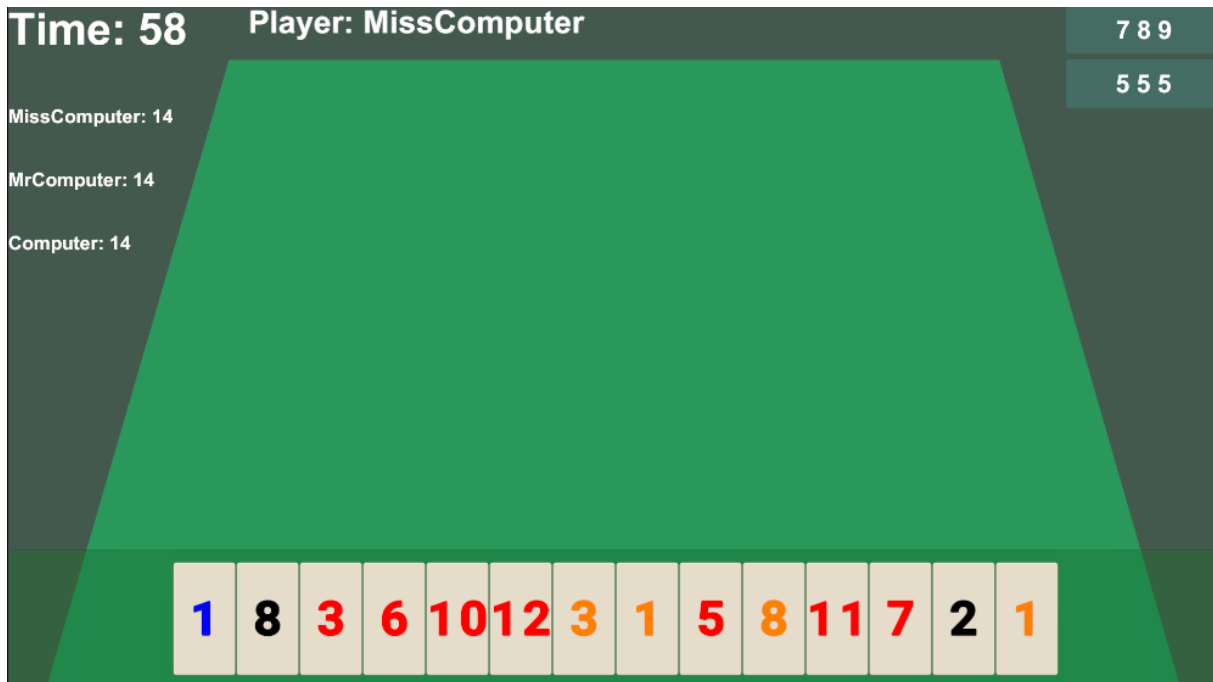


Rysunek 4.1: Menu gry

Na ekranie widać cztery przyciski, każdy z nich wykonuje co innego:

- **Play with computer players** - jest to rozgrywka, w której biorą udział trzej gracze komputerowi z predefiniowanymi ruchami wraz z użytkownikiem. Tryb służy jedynie do potencjalnego zapoznania się z grą i jej mechanikami.
- **Play with AI and computer players** - tryb gry, w którym użytkownik gra wraz z dwoma graczami komputerowymi oraz jednym graczem komputerowym z algorytmami sztucznej inteligencji. Jest to docelowy tryb projektu.
- **AI vs computer players** - w tym trybie gry jest jeden gracz z algorytmami sztucznej inteligencji oraz trzej gracze komputerowi z predefiniowanymi ruchami. Ten tryb służy tylko do nauki modelu.
- **Play with AI** - ten tryb gry jest grą jeden na jeden, gdzie użytkownik gra przeciwko komputerowemu graczowi z uczeniem maszynowym. Ten tryb służy do walidacji nauki danego modelu.

- **Quit game** - przycisk służący do opuszczenia gry.



Rysunek 4.2: Przykładowy ekran początku gry

Na obrazku 4.2 można zobaczyć przykładowy ekran rozpoczęcia gry. Gra rozpoczyna się z każdym graczem posiadającym po czternaście płytek na swoim stojaku.

Nad mapą widać napis zaprezentowany na rysunku 4.3.



Rysunek 4.3: Aktualna tura: Gracz X

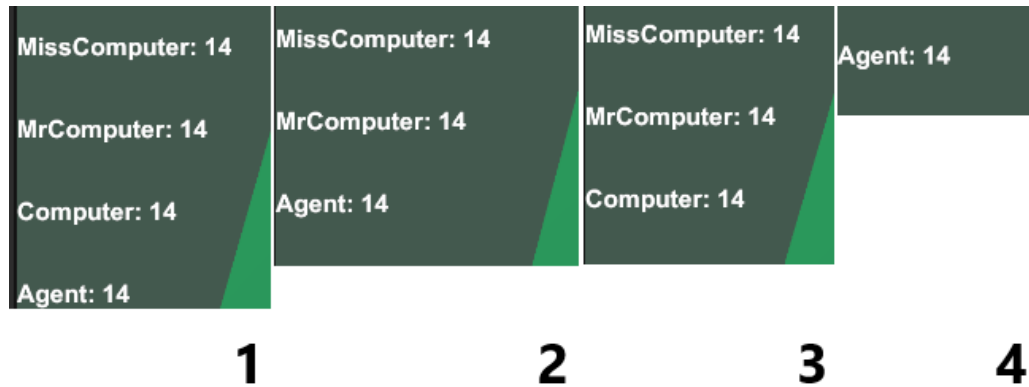
Napis ten informuje użytkownika, który gracz w danej chwili wykonuje ruch. W miejscu X mogą pojawić się różne napisy:

- Computer, MissComputer, MrComputer - są to wyznaczniki, że daną turę wykonuje gracz komputerowy o predefiniowanych ruchach.
- You - informacja, że to użytkownik może wykonać ruch.
- Agent - oznacza to, że gracz komputerowy z algorytmem uczenia maszynowego ma turę w danym momencie.

W lewym górnym rogu rysunku 4.2 znajduje się informacja o czasie w aplikacji. Każda tura trwa pewną ilość czasu, jeśli dany gracz nie zakończy swojej tury przed czasem,

wszystkie jego ruchy zostają cofnięte, oraz dostaje nową płytkę z banku. Jedna tura zawierająca ruch trwa 60 sekund.

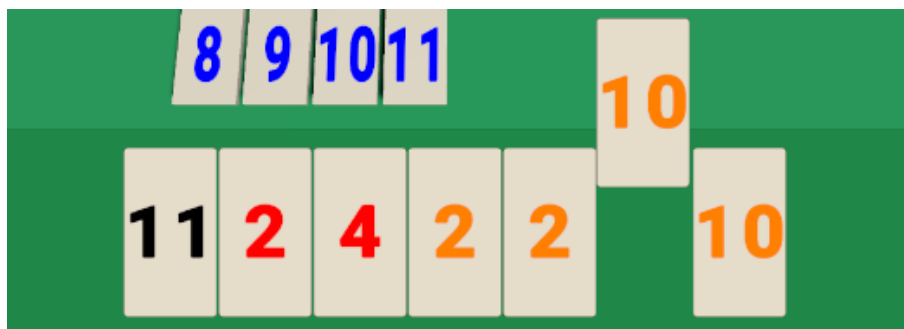
Z lewej strony ekranu znajduje się lista graczy komputerowych. Dostarcza ona informacje, ile ma płytek każdy z graczy. Lista graczy różni się w zależności od trybu gry, co można zauważyć na rysunku 4.4.



Rysunek 4.4: widok listy graczy

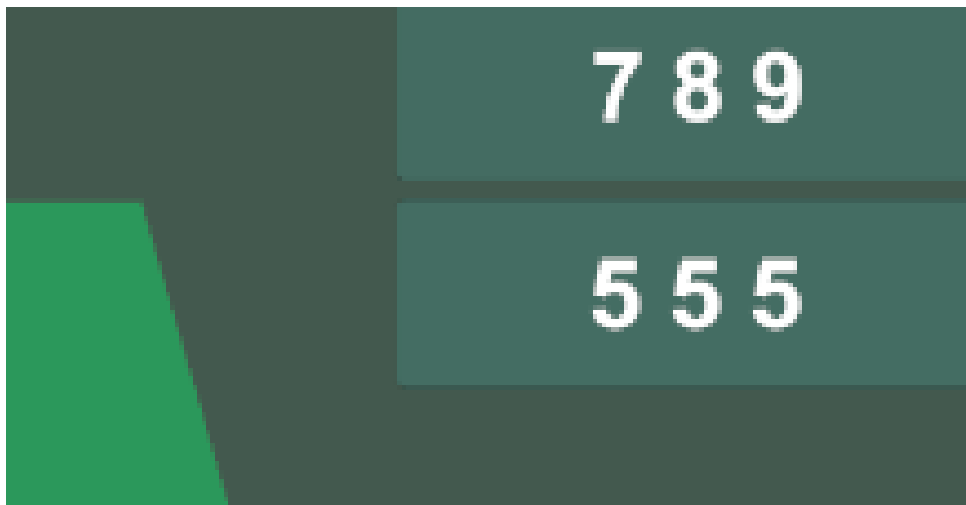
1. Jest to lista graczy w trybie nauki. Znajdują się tutaj trzej gracze komputerowi z predefiniowanymi ruchami oraz jeden agent. Lista w tym trybie jest jedynie poglądowa dla użytkownika.
2. To lista graczy z trybu gry użytkownik przeciwko dwóm graczom komputerowym z predefiniowanymi ruchami oraz jednemu graczowi komputerowemu z algorytmami uczenia maszynowego.
3. Jest to lista z trybu gracz przeciwko trzem graczom komputerowym z predefiniowanymi ruchami.
4. Ta lista jest z trybu gry użytkownik przeciwko gracz komputerowy z algorytmem uczenia maszynowego.

Użytkownik ma możliwość interakcji z płytkami w stojaku, widocznym na rysunku 4.5. Przytrzymanie wybranej płytki pozwala na jej przesuwanie w prawo bądź lewo w celu ułożenia jej w konkretnej sekwencji.



Rysunek 4.5: widok stojaku z płytkami

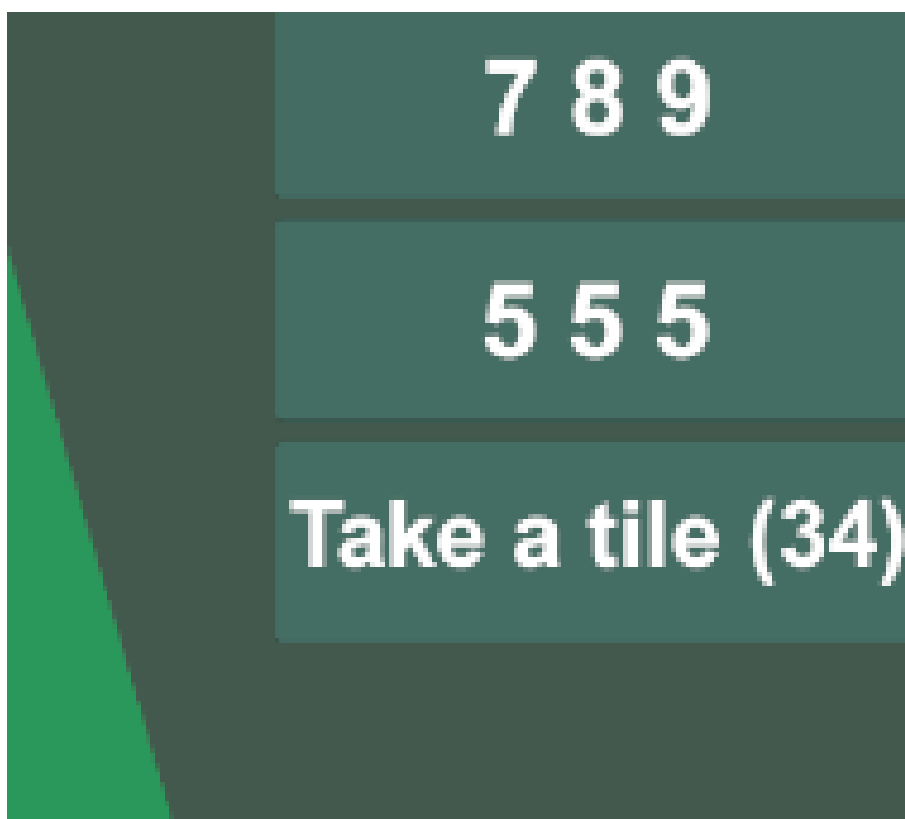
W prawym górnym rogu ekranu znajdują się przyciski. W zależności od momentu rozgrywki:



Rysunek 4.6: przyciski poza turą

Gdy nie ma tury użytkownika, wygląd przycisków wygląda jak na rysunku 4.6:

- pierwszy przycisk służy do sortowania względem serii.
- drugi przycisk służy do sortowania względem grup.

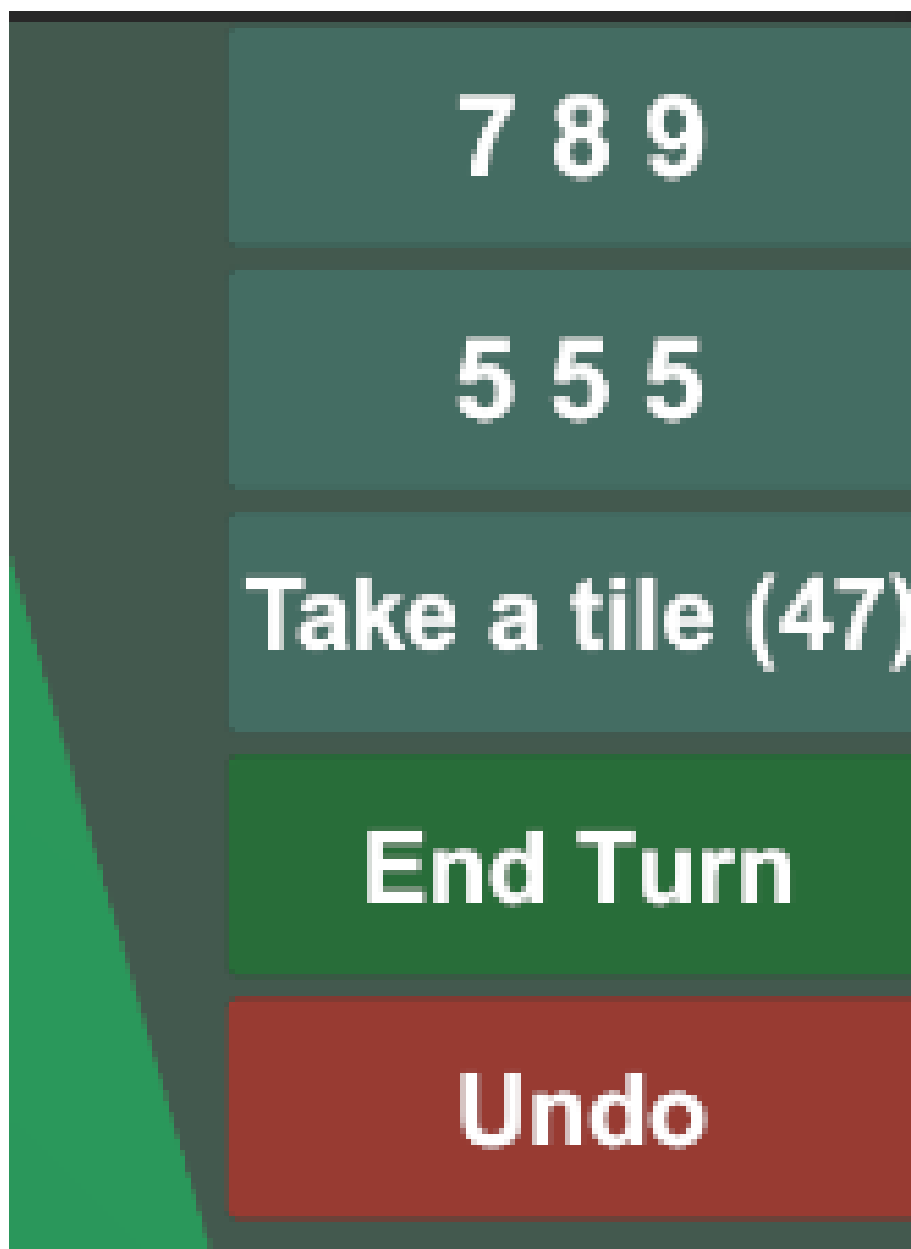


Rysunek 4.7: przyciski przed wykonaniem ruchu

Gdy użytkownik ma swoją turę, ale jeszcze nie wykonał ruchu, pojawia się dodatkowy przycisk **Take a tile (34)**, widoczny na rysunku 4.7, który pozwala graczowi na pobranie płytki, wraz z czym kończy turę. Liczba w nawiasach w przycisku informuje, ile zostało płytek w głównym banku. Jeśli został użyty po wykonaniu ruchu na mapie, wszystkie manipulacje zostaną cofnięte.

Przycisk **End Turn**, widoczny na rysunku 4.8 wykona sprawdzenie, czy wszystkie sekwencje na mapie zawierają co najmniej trzy płytki. Jeśli funkcja wykryje błąd, tura nie zakończy się oraz zostanie pokazany komunikat o niepoprawnym zakończeniu tury.

Przycisk **Undo** cofa wszystkie zmiany i wyłożenia, jakie użytkownik wykonał w trakcie danej tury.



Rysunek 4.8: przyciski po wykonaniu ruchu

Jeśli gracz spróbuje zakończyć turę przyciskiem **End Turn**, podczas gdy na mapie co najmniej jedna sekwencja nie zawiera minimalnie trzech płytek, na ekranie użytkownika wyświetli się ten komunikat widoczny na rysunku 4.9.



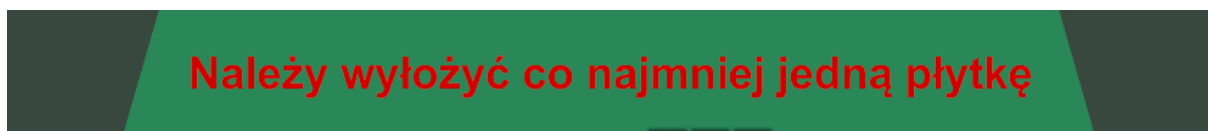
Rysunek 4.9: komunikat o źle zakończonej mapie

Jeśli gracz wykonuje swoją pierwszą turę, ale suma sekwencji, jakie wyłożył, nie jest równa bądź większa trzydzieści, to na ekranie pokaże się komunikat widoczny na rysunku 4.10.



Rysunek 4.10: komunikat o źle zakończonej pierwszej turze

Gdy gracz już zakończył swoją pierwszą turę i w trakcie kolejnej tury wykona manipulacje sekwencjami na mapie nie wykładając przy tym ani jednej płytki ze swojego stojaka, pokaże się komunikat zaprezentowany na rysunku 4.11.



Rysunek 4.11: komunikat braku wyłożonej płytki

Nie dozwolone jest, na daną turę, manipulowanie mapą bez wyłożenia nowej płytki. Jeżeli gracz zdecyduje się zabrać z planszy płytkę, która została położona przez niego bądź kogoś innego w innej turze, na ekranie pojawi się komunikat widoczny na rysunku 4.12.



Rysunek 4.12: komunikat przy niedozwolonym usuwaniu płytki

Gdy pierwszy gracz wyłoży wszystkie płytki na mapę, pokaże się ekran zakończenia gry, przedstawionym na rysunku 4.13.



Rysunek 4.13: ekran końca gry

Na rysunku 4.13 widać hierarchię graczy w zależności od ich wyniku. Każdy gracz w hierarchii, pod zwycięzcą, ma w nawiasie zapisany wynik sumy płytek, które miał na swoim stojaku pod koniec gry. W takiej sytuacji wygrywa gracz, który nie ma już żadnych płytek na stojaku.

Jeśli gra jednak zakończy się brakiem płytek w głównym banku, ekran końca gry będzie wyglądać jak na rysunku 4.14.



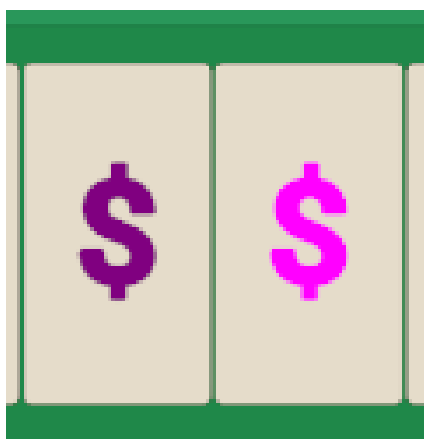
Rysunek 4.14: ekran końca gry przy pustym banku

W takiej sytuacji, każdy gracz ma wyliczoną sumę płytek na swoim stojaku. Wygrywa gracz, którego suma punktów jest najmniejsza.

Na ekranie końca gry, oprócz wyników graczy, znajdują się dwa przyciski:

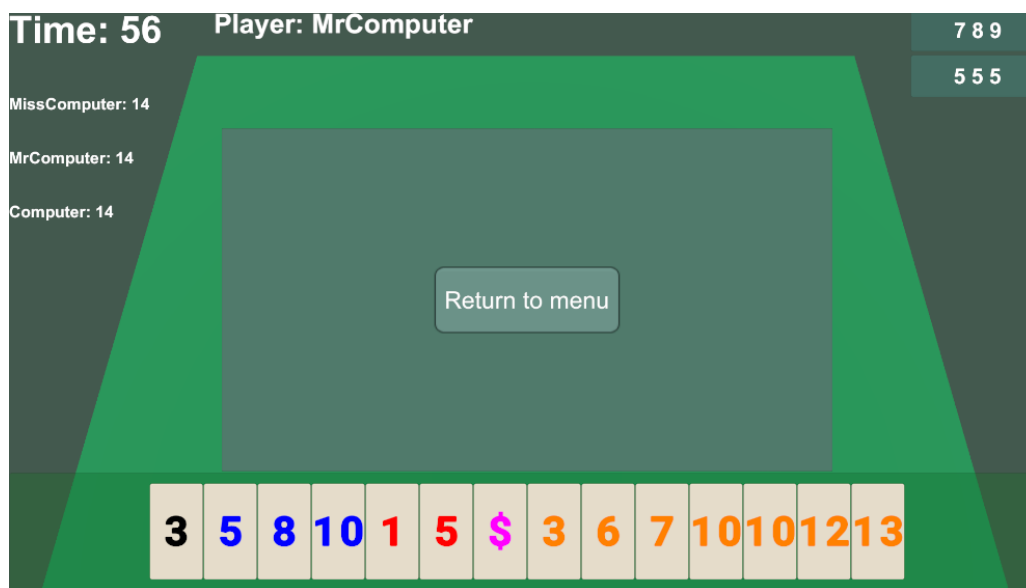
- **Return to menu** - jego użycie przeniesie użytkownika do menu głównego gry.
- **Play again** - wykorzystanie tego przycisku pozwoli graczowi wzięcie udziału w nowej rozgrywce w tym samym trybie.

Każda płytką w grze odpowiada obiektom wykorzystywanym w oryginalnej grze planszowej, oprócz płytek jokerów. Rysunek 4.15 prezentuje wygląd jokerów w grze.



Rysunek 4.15: jokerzy

W trakcie gry istnieje możliwość jej zatrzymania, na przykład gdy użytkownik musi na chwilę odejść od urządzenia lub chce zmienić tryb gry bez konieczności czekania na zakończenie rozgrywki. W takim przypadku wystarczy nacisnąć przycisk **Esc**. Po jego naciśnięciu, pojawi się ekran, który jest widoczny na rysunku 4.16.

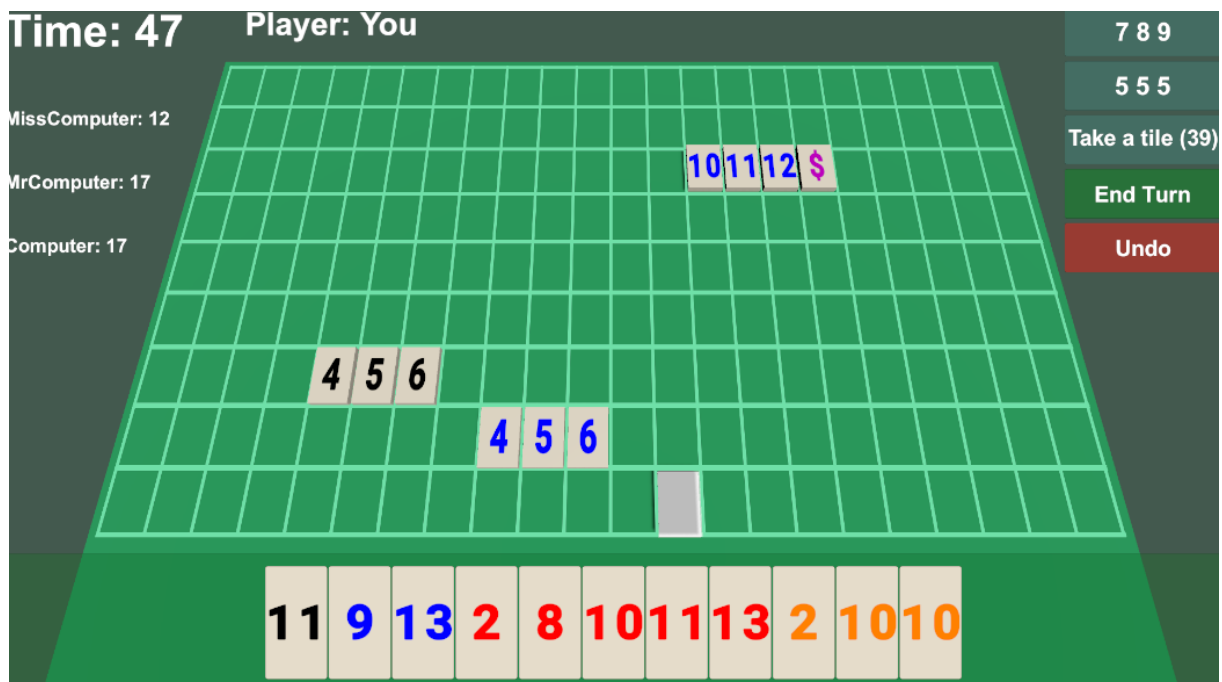


Rysunek 4.16: Ekran zatrzymania gry

W tym momencie gra zostaje wstrzymana, a dalsza rozgrywka nie jest możliwa. Jeśli użytkownik zdecyduje się zakończyć grę, może to zrobić, klikając przycisk **Return to menu**, co spowoduje powrót do głównego ekranu gry i zakończenie bieżącej sesji. Jeżeli gracz chce wznowić rozgrywkę, wystarczy, że naciśnie ponownie klawisz **Esc**, co pozwoli na kontynuację gry od miejsca, w którym została przerwana.

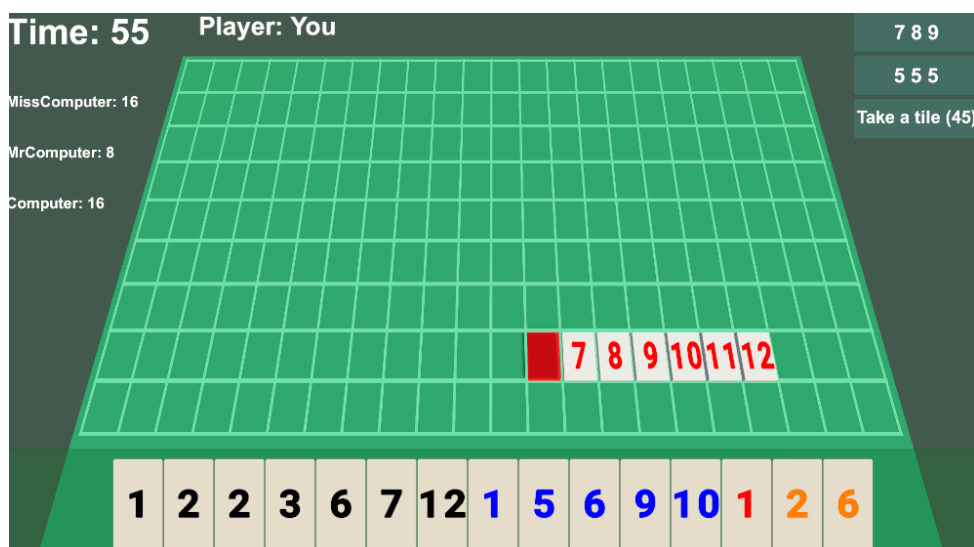
4.3.3 Interakcja z płytkami gry

Po wybraniu trybu gry, użytkownik zostaje przeniesiony do mapy rozgrywki. Jego głównym celem jest jak najszybsze wyłożenie płytek ze swojego stojaka. W trakcie interakcji z mapą pojawia się siatka, która pomaga graczowi zorientować się, gdzie dokładnie można umieścić płytkę. Aby położyć płytkę na mapie, należy najpierw kliknąć lewym przyciskiem myszy na wybraną płytkę, a następnie przesunąć kursor w kierunku miejsca, w którym gracz chce ją umieścić. Na rysunku 4.17 widać obiekt, podobny do płytki. Gracz dzięki temu widzi, gdzie płytka będzie położona. Aby zakończyć akcję, wystarczy nacisnąć lewym przyciskiem myszy punkt na mapie.



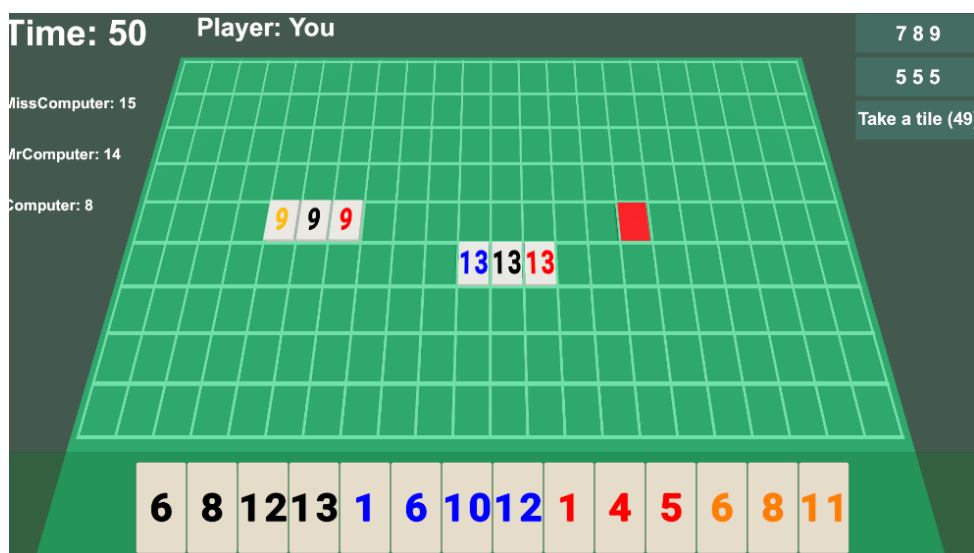
Rysunek 4.17: Kładzenie płytki na mapę

Jeśli gracz przeciągnie kursorem myszy na pole, na którym nie może położyć płytki, obiekt zostanie pokazany na czerwono tak jak widać na rysunku 4.18.



Rysunek 4.18: Złe położenie płytki

Aby usunąć płytkę z mapy, należy wcisnąć klawisz D, a następnie kliknąć lewym przyciskiem myszy na płytkę, którą chce się usunąć. Ta akcja usuwa płytkę z mapy i przywraca ją do stojaka gracza. Użytkownik może usunąć jedynie płytki wyłożone w trakcie trwania tur. Gracz, chcąc usunąć płytkę z mapy zobaczy czerwony obiekt jako kursor, jak na rysunku 4.19. Oznacza on, że nie można usunąć obiekt z miejsca, w którym obiektu nie ma. Obiekt zmienia kolor na biały, gdy zostanie umiejscowiony na pozycji, gdzie znajduje się płytką



Rysunek 4.19: wygląd usuwania płytki

Gdy gracz chce przesunąć płytkę na mapie, musi najpierw nacisnąć prawym przyciskiem myszy na wybraną płytkę. Następnie wybrać nową pozycję na mapie i nacisnąć lewy przycisk myszy na wybranej pozycji. W trakcie przesuwania płytki pojawia się obiekt tak samo, jak przy układaniu nowej płytki

Jeśli gracz zdecyduje się zaprzestać wykonywania którejś akcji jak wykładania czy przesuwania, musi nacisnąć klawisz Q

4.3.4 Predefiniowani gracze komputerowi

Gracze komputerowi z predefiniowanymi zachowaniami posiadają zaimplementowane funkcje obsługujące ich zachowania. Funkcjonalność gracza przebiega w następującym schemacie:

1. Analizuje płytki na swoim stojaku i poszukuje możliwych serii do położenia. Jeżeli to jest pierwsza tura gracza, to sprawdzi czy suma punktów wszystkich serii wynosi co najmniej 30. Jeśli warunki zostaną spełnione, kładzie serie w losowym miejscu na mapie.
2. Analizuje płytki na swoim stojaku w poszukiwaniu wszystkich możliwych grup do położenia. Jeżeli jest to pierwsza tura gracza, to sprawdzi czy suma punktów wszystkich wytypowanych grup wynosi co najmniej 30. Jeśli warunek zostanie spełniony, płytki zostaną położone na mapie.
3. Jeżeli gracz zakończył już pierwszą turę, to po wyłożeniu wszystkich możliwych serii i grup ze stojaka, może dokładać płytki na mapę do istniejących ciągów. Gracz sprawdza wszystkie wolne pozycje obok położonych sekwencji. Jeżeli wybrana płytka może zostać dołożona do ciągu, gracz położy płytkę.
4. Gracz może zakończyć turę. Najpierw jednak sprawdzi czy ilość płytek w jego stojaku jest taka sama jak na końcu tury. Jeżeli tak, to gracz komputerowy dobierze kolejną płytkę, jeśli nie to tura zakończy się.

4.3.5 Nauka gracza komputerowego

Użytkownik po wykonaniu akcji podanych w rozdziale **Aktywacja** może zaobserwować uruchomienie się środowiska nauki modelu.



Rysunek 4.20: Włączanie nauczania w wierszu poleceń

Na rysunku 4.20 widać uruchomione środowisko, które wyczeka uruchomienia symulacji w unity na konkretnym porcie przesyłowym. Pokazują się również parametry modelu. Bezpośrednio po uruchomieniu symulacji w Unity, w wierszu poleceń wyświetli się lista pokazana na rysunku 4.21. Jest to spis parametrów, jakie model ma ustawione przy nauce. Po uruchomieniu nauki, użytkownik może zauważyć, że w unity uruchomiła się symulacja. Gracz komputerowy z uczeniem maszynowym zacznie wykonywać losowe ruchy, które mu zostały przypisane, po pewnym czasie jednak będą już oparte o decyzyjność agenta. Podejmowane decyzje modelu są oparte na karach i nagrodach jakie otrzymuje w trakcie rozgrywki. Każdy model ma określoną ilość trwania ruchów, zanim zostaną zaktualizowane wagi modelu, co za tym idzie, podejmowanie już bardziej przemyślanych decyzji. Użytkownik ma możliwość widzieć w trakcie nauki, jakie model dostaje nagrody i postęp z każdą aktualizacją, co widać na rysunku 4.22.

```

_C.set_default_tensor_type(t)
[INFO] Listening on port 5004. Start training by pressing the Play button in the Unity Editor.
[INFO] Connected to Unity environment with package version 2.0.1 and communication version 1.5.0
[INFO] Connected new brain: PlayRummikub?team=0
[INFO] Hyperparameters for behavior name PlayRummikub:
trainer_type: ppo
hyperparameters:
  batch_size: 512
  buffer_size: 8192
  learning_rate: 0.00025
  beta: 0.0005
  epsilon: 0.05
  lambda: 0.95
  num_epoch: 8
  shared_critic: False
  learning_rate_schedule: linear
  beta_schedule: constant
  epsilon_schedule: constant
network_settings:
  normalize: True
  hidden_units: 256
  num_layers: 3
  vis_encode_type: simple
  memory: None
  goal_conditioning_type: hyper
  deterministic: False
reward_signals:
  extrinsic:
    gamma: 0.995
    strength: 1.0
  network_settings:
    normalize: False
    hidden_units: 128
    num_layers: 2
    vis_encode_type: simple
    memory: None
    goal_conditioning_type: hyper
    deterministic: False
init_path: None
keep_checkpoints: 5
checkpoint_interval: 500000
max_steps: 20000000
time_horizon: 128
summary_freq: 5000
threaded: False
self_play: None
behavioral_cloning: None

```

Rysunek 4.21: Parametry modelu w wierszu poleceń

Parametry modelu, widoczne w wierszu poleceń, mogą być modyfikowane przez użytkownika. W tym celu należy stworzyć plik `.yaml`, schemat pliku został opisany w rozdziale **Specyfikacja wewnętrzna**. Nauka modelu polega na włączaniu symulacji gry w pętli, co daje graczowi czas na analizę.

```
[INFO] PlayRummikub. Step: 5000. Time Elapsed: 67.101 s. Mean Reward: -269.500. Std of Reward: 0.000. Training.
[INFO] PlayRummikub. Step: 10000. Time Elapsed: 127.862 s. Mean Reward: -269.000. Std of Reward: 0.000. Training.
[INFO] PlayRummikub. Step: 15000. Time Elapsed: 172.568 s. Mean Reward: -319.300. Std of Reward: 0.000. Training.
[INFO] PlayRummikub. Step: 20000. Time Elapsed: 234.264 s. Mean Reward: -180.950. Std of Reward: 55.250. Training.
[INFO] PlayRummikub. Step: 25000. Time Elapsed: 295.619 s. Mean Reward: -470.800. Std of Reward: 0.000. Training.
[INFO] PlayRummikub. Step: 30000. Time Elapsed: 339.878 s. Mean Reward: -358.000. Std of Reward: 0.000. Training.
[INFO] PlayRummikub. Step: 35000. Time Elapsed: 400.572 s. Mean Reward: -221.500. Std of Reward: 0.000. Training.
[INFO] PlayRummikub. Step: 40000. Time Elapsed: 445.334 s. Mean Reward: -232.200. Std of Reward: 48.500. Training.
[INFO] PlayRummikub. Step: 45000. Time Elapsed: 506.442 s. Mean Reward: -343.300. Std of Reward: 0.000. Training.
[INFO] PlayRummikub. Step: 50000. Time Elapsed: 567.557 s. Mean Reward: -306.300. Std of Reward: 0.000. Training.
[INFO] PlayRummikub. Step: 55000. Time Elapsed: 613.190 s. Mean Reward: -279.625. Std of Reward: 14.375. Training.
[INFO] PlayRummikub. Step: 60000. Time Elapsed: 674.021 s. Mean Reward: -316.600. Std of Reward: 0.000. Training.
```

Rysunek 4.22: Aktualizacje modelu w wierszu poleceń

W trakcie nauki oraz po jej zakończeniu, użytkownik może przejrzeć przebieg nauki na wygenerowanych wykresach. W tym celu musi uruchomić środowisko python w konsoli wpisując komendę:

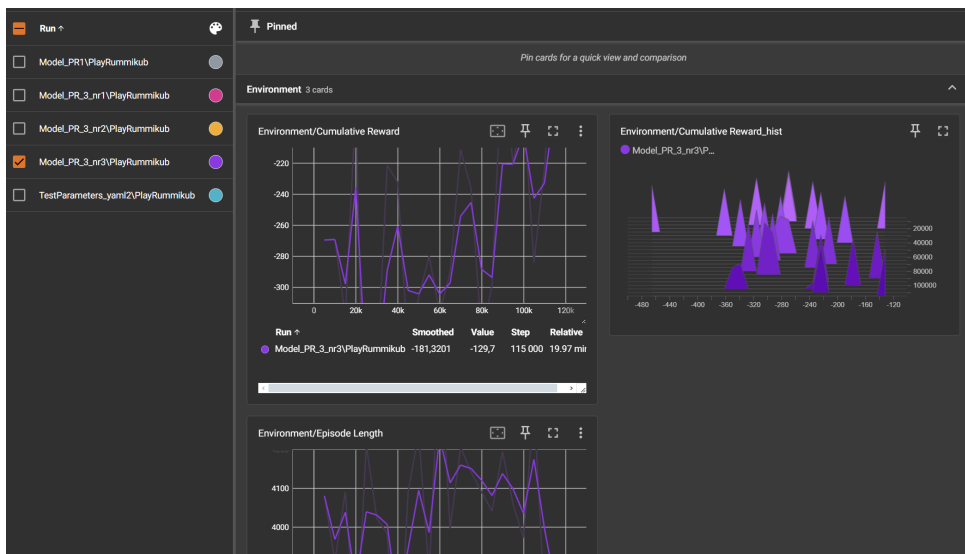
```
tensorboard --logdir results
```

Konsola wtedy wyświetli adres strony zaznaczony na rysunku 4.23, na którym zostaną wygenerowane wykresy.

```
(venv) C:\Users\jwiec\Desktop\Rummikub-with-ai\RummiAI>tensorboard --logdir results
TensorFlow installation not found - running with reduced feature set.
Serving TensorBoard on localhost; to expose to the network, use a proxy or pass --bind_all
TensorBoard 2.18.0 at http://localhost:6006/ (Press CTRL+C to quit)
```

Rysunek 4.23: Uruchomienie strony z wynikami

Wpisanie podanego adresu, wyświetli użytkownikowi stronę pokazaną na rysunku 4.24. Można tam wyświetlić wykresy wszystkich wyuczonych modeli, które są dostępne dla środowiska Python.



Rysunek 4.24: Strona z wynikami

Rozdział 5

Specyfikacja wewnętrzna

Specyfikacja wewnętrzna opisuje szczegóły techniczne implementacji projektu. Projekt został zrealizowany w środowisku Unity, jego kluczowe komponenty zostały zaimplementowane w C#. To pozwoliło zaimplementować logikę gry, obiekty wykorzystywane w grze oraz gracza komputerowego z algorytmami uczenia maszynowego.

5.1 Struktura projektu

W aplikacji zostały zaimplementowane obiekty oraz klasy konieczne do obsługi logiki rozgrywki, interakcji z obiektami oraz wyglądu gry. Dzielą się one na główne moduły:

- Moduł planszy gry - Składa się na obiekt mapy gry, jest ona widoczna dla użytkownika oraz gracze mogą wchodzić z nią w interakcje. W jej skład wchodzi obiekty siatki, do układania płytek na mapę, prefabrykatowy płytek 3D, wizualizacja siatki jako widocznej struktury oraz interakcja przemieszczania czy dodawania płytek. Główne klasy składające się na moduł to: IPlacementState, ObjectPlacer.
- Moduł interfejsu użytkownika - interfejs gry, z którym użytkownik może wchodzić w interakcje. W jej skład wchodzi napisy, przyciski oraz prefabrykatowy płytek 2D. Użytkownik widzi w interfejsie płytki na swoim stojaku, ile płytek mają inni gracze, czyja tura jest oraz ile czasu pozostało do zakończenia bieżącej tury. Główne klasy składające się na moduł to: Draggable, GameTurnManager, TakeTile.
- Moduł kontrolera - główny kontroler gry, łączy on całą logikę gry z wizualną częścią aplikacji. Klasy składające się na moduł: GameController, TurnController.
- Moduł logiki gry - moduł odpowiadający za kalkulację oraz walidację gry względem zasad gry planszowej. W jej skład wchodzi funkcjonalności walidacji w trakcie interakcji z płytkami na mapie, sprawdzenie poprawności planszy podczas zakończenia tury. Główne klasy składające się na moduł to: BoardDictionary, Tile.

- Moduł użytkownika - odpowiada za przechowywanie danych o płytkach, jakie gracz posiada na stojaku. Klasa wchodząca w skład modułu: Player.
- Moduł gracza komputerowego - implementuje mechanizmy działania takie jak funkcjonalność predefiniowanych ruchów oraz zarządzanie stojakiem. Główną klasą modułu jest: ComputerPlayer.
- Moduł uczenia maszynowego - implementuje gracza komputerowego o algorytmach uczenia maszynowego, wdraża jego mechaniki w środowisko gry oraz implementuje kontrolę przypisywania nagród. Głównymi klasami modułu są: PlayerAI, Agent.

5.2 Moduł planszy gry

1. IPlacementState - własny interfejs do wykonywania interakcji użytkownika z mapą gry. Metodami zaimplementowanymi w interfejsie są:
 - void OnAction(Vector3Int gridPosition) - Metoda, wykonująca się po kliknięciu prawym przyciskiem myszy.
 - void UpdateState(Vector3Int gridPosition) - funkcja zmieniająca pozycję obiektu grid na bieżąco w zależności od pozycji kursowa myszy
 - void EndState() - Metoda kończąca daną akcję.
2. PlacementState - dziedziczy metody po IPlacementState, służy do kładzenia nowej płytki na mapę.
 - public void OnAction(Vector3Int gridPosition) - Metoda, po kliknięciu prawego przycisku myszy na wybranej pozycji, tworzy obiekt na mapie.
 - public void UpdateState(Vector3Int gridPosition) - funkcja zmieniająca pozycję obiektu na bieżąco w zależności od pozycji kursowa myszy.
 - public void EndState() - Metoda kończąca dodawanie obiektu na mapę, usuwa się obiekt wizualny.
 - private bool IsPositionInRange(Vector3 position, Vector3 minRange, Vector3 maxRange) - Metoda zabezpieczająca, aby gracz nie mógł postawić płytki poza mapą rozgrywki.
 - private bool CheckPlacementValidity(Vector3Int gridPosition) - Funkcja sprawdza, czy wybrana pozycja jest pusta bądź położenie na niej płytki jest zgodne z zasadami.
3. RemovingState - dziedziczy metody po IPlacementState, służy do usuwania płytek z mapy.

- `public void OnAction(Vector3Int gridPosition)` - Metoda, po kliknięciu prawego przycisku myszy na wybranej pozycji, usuwa wybrany obiekt na mapie.
 - `public void UpdateState(Vector3Int gridPosition)` - funkcja zmieniająca pozycję obiektu grid na bieżąco w zależności od pozycji kursora myszy.
 - `public void EndState()` - Metoda kończąca usuwanie obiektu z mapy, usuwa się obiekt wizualny.
 - `private bool CheckIfSelectionIsValid(Vector3Int gridPosition)` - Funkcja sprawdza, czy wybrana pozycja zawiera płytkę, którą można usunąć.
4. `MovingState` - dziedziczy metody po `IPlacementState`, służy do przesuwania płytek na mapie.
- `public void OnAction(Vector3Int gridPosition)` - Metoda, po kliknięciu prawego przycisku myszy na wybranej pozycji, przesuwa wybrany obiekt na mapie.
 - `public void UpdateState(Vector3Int gridPosition)` - funkcja zmieniająca pozycję obiektu grid na bieżąco w zależności od pozycji kursora myszy.
 - `public void EndState()` - Metoda kończąca przesuwanie obiektu po mapie, usuwa się obiekt wizualny.
 - `private bool CheckIfSelectionIsValid(Vector3Int gridPosition)` - Funkcja sprawdza, czy wybrana pozycja zawiera płytkę, którą można przesunąć.
 - `private bool IsPositionInRange(Vector3 position, Vector3 minRange, Vector3 maxRange)` - Metoda zabezpieczająca, aby gracz nie mógł przestawić płytkę poza mapę rozgrywki.
5. `PreviewSystem` - Tworzy obiekt wizualny dla gracza, przemieszcza się po mapie wraz z kursorem użytkownika. Tworzony obiekt zmienia kolor w zależności, czy interakcje można wykonać czy nie.
- `internal void StartShowingRemovePreview()` - tworzenie obiektu dla akcji usuwania płytki z mapy.
 - `internal void StartShowingPlacementPreview(GameObject prefabe, Vector2Int size)` - funkcja tworząca obiekt wizualny na mapie dla funkcji dodawania i przesuwania obiektu.
 - `private void PrepareCursor(Vector2Int size)` - ustala wielkość tworzonego obiektu wizualnego.
 - `private void PreparePreview(GameObject previewObject)` - funkcja przygotowująca obiekt na scenie.
 - `public void StopShowingPreview()` - Funkcja do usunięcia obiektu wizualnego z mapy.

- `public void UpdatePosition(Vector3 position, bool validity)` - metoda służąca aktualizacji pozycji obiektu wizualnego.
- `private void ApplyFeedbackToPreview(bool validity)` - funkcja nakładająca kolor obiektowi, w zależności zmiennej `validity`. Wykorzystywana w akcjach dodawania i przesuwania płytek.
- `private void ApplyFeedbackToCursor(bool validity)` - funkcja nakładająca kolor obiektowi, w zależności zmiennej `validity`. Wykorzystywana w akcji usuwania płytek z mapy.
- `private void MoveCursor(Vector3 position)` - zmiana pozycji kursora przy usuwaniu płytek.
- `private void MovePreview(Vector3 position)` - zmiana pozycji obiektu przy dodawaniu i przesuwanie obiektów.

6. `InputManager` - klasa do wykrywania pozycji myszy na mapie.

- `public bool isPointerOverUI()` - funkcja służąca sprawdzeniu czy kursor wykonujący akcje znajduje się nad ui czy też nie.
- `public Vector3 GetSelectedMapPosition()` - Przekazanie pozycji myszki w zależności od tego, czy kursor myszy znajduje się nad mapą czy poza nią.

7. `ObjectPlacer` - Klasa, która tworzy obiekty na mapie.

- `public void ClearplacedGameObjects()` - metoda usuwająca wszystkie obiekty na planszy.
- `public void SetPlacedGameObjectsCopy()` - nadpisuje kopie obiektów na mapie.
- `public void RemoveObjectsNotInCopy()` - metoda usuwa obiekty na mapie, których nie ma w liście kopii, używana przy cofaniu akcji.
- `public bool ListAreEqual()` - metoda porównuje dwie listy obiektów, listę podstawową i kopię.
- `public int PlacedObject(GameObject prefab, Vector3 position, ref Tile tile, Grid grid)` - metoda do tworzenia płytki na mapie.
- `internal void RemoveObjectAt(int gameObjectIndex)` - Metoda usuwająca obiekt z mapy.
- `internal void MoveObjectTo(int gameObjectIndex, Vector3 newPosition)` - metoda przesuująca obiekt na mapie.

5.3 Moduł interfejsu użytkownika

1. Draggable - Klasa dziedziczy po interfejsach `IBeginDragHandler`, `IDragHandler`, `IEndDragHandler` dostarczanych przez środowisko Unity. Służy do interakcji przesuwania płytek 2D w stojaku przez użytkownika.
 - `public void OnBeginDrag(PointerEventData eventData)` - funkcja rozpoczynająca przesuwanie obiektu, odziedziczona od `IBeginDragHandler`.
 - `public void OnDrag(PointerEventData eventData)` - funkcja aktualizująca, odziedziczona od `IDragHandler`.
 - `public void OnEndDrag(PointerEventData eventData)` - funkcja kończąca interakcje z obiektem, odziedziczona od `IEndDragHandler`.
2. GameTurnManager - klasa do aktualizowania wyglądu interfejsu gry.
 - `public void SetUIText()` - ustawianie napisu gracza, który ma turę.
 - `public void SetPlayersTileCountUI()` - aktualizowanie ilości płytek każdego gracza.
 - `public void ChangeTurn()` - informacja o zakończeniu tury.
 - `public void SetTimeUI()` - aktualizacja czasu na ekranie.
3. TakeTile - Klasa, tworząca płytki na stojaku gracza oraz przechwytyująca interakcje użytkownika z interfejsem.
 - `public void SetStartTile()` - metoda tworząca obiekty płytek na początku gry.
 - `public void OnButtonClick(Tile tile, ref int idx)` - metoda przekazująca informacje, jaka płytka została wybrana do położenia.
 - `public void takeNewTile()` - obsługuje przycisk dodania nowej płytki do stojaka użytkownika.
 - `public void DisableAllButtons()` - funkcja do anulowania możliwości interakcji z płytkami, gdy gracz jest poza swoją turą.
 - `public void EnableAllButtons()` - przywrócenie użytkowania płytek w stojaku.
4. MainMenu - klasa do interakcji gracza z menu gry.
 - `public void PlayWithCP()` - uruchamia tryb "gra użytkownika z trzema graczami komputerowymi z predefiniowanymi zachowaniami".
 - `public void AIvsCP()` - uruchamia tryb do nauki agenta.
 - `public void PlayervsAIvsCP()` - uruchamia tryb gry "użytkownik przeciwko agentowi i 2 graczom komputerowym z predefiniowanymi funkcjami".

- `public void PlayWithAI()` - uruchamia tryb "użytkownik przeciwko agent"
- `public void ReturnToMenu()` - metoda powrotu do menu głównego gry.
- `public void QuitGame()` - metoda do opuszczenia gry.

5.4 Moduł kontrolera

1. `TurnController` - Kontroler tur gry, sprawdza czy wielkości sekwencji na mapie są poprawne, przekazuje informacje o błędach do interfejsu.

- `public void NewTurn()` - funkcja sprawdzająca warunki poprawnego zakończenia tury. Jeśli nie wykryje błędu, to aktualizuje status płytek i kończy turę.
- `public bool CheckMap()` - metoda, która sprawdza długości sekwencji na mapie.
- `public void Restore3DMap()` - po wykonaniu akcji "undo", metoda przywraca mapę sprzed aktualizacji.
- `public void CancelMove()` - metoda anulowania wszystkich manipulacji wykonanych na mapie.
- `public bool MapContents()` - metoda sprawdzająca, czy na daną turę mapa została zmodyfikowana.

2. `GameController` - Główna klasa kontrolera, zarządza danymi z głównego banku gry.

- `void CreateMainBank(ref List<Tile> tiles)` - metoda tworząca i dodająca obiekty typu `Tile` do głównego banku gry. Metoda jest zawsze wywoływana na początku gry.
- `public void NewTurn()` - metoda aktualizująca obiekt `BoardGameDictionary` po zakończeniu tury.
- `public void StopGame()` - metoda zatrzymująca grę i otwierająca okno zatrzymania gry
- `public void EndGame()` - metoda przekazująca dane graczy, jak ich wyniki, do ekranu zakończenia gry.

3. `PlacementSystem` - klasa kontrolująca akcje wykonywane na mapie przez użytkownika.

- `public void StartPlacement(int ID, ref Tile tile, ref int index)` - metoda tworząca obiekt `PlacementState` w celu tworzenia nowej płytki na mapie.
- `public void StartRemoving()` - metoda tworząca obiekt `RemovingState` w celu usunięcia istniejącej płytki z mapy.

- `public void StartMowing()` - metoda tworząca obiekt `MovingState` w celu przesunięcia pojedynczej płytki na mapie.
- `private void PlaceStructure()` - metoda pobierająca informacje pozycje kursora na mapie.
- `public void StopPlacement()` - metoda usuwająca jeden z obiektów, kończąc akcję.

5.5 Moduł logiki gry

1. `GridData` - klasa do przechowywania danych dla klasy `ObjectPlacer` takie jak pozycja względem siatki oraz indeks w liście obiektu `ObjectPlacer`.
 - `internal int getRepresentationIndex(Vector3Int gridPosition)` - pobranie indeksu obiektu na bazie podanej pozycji w celu interakcji z konkretnym obiektem na mapie.
 - `public void AddObjectAt(Vector3Int gridPosition, int placedObjectIndex)` - dodawanie nowego obiektu do słownika
 - `internal void RemoveObjectAt(Vector3Int gridPosition)` - usunięcie obiektu z słownika.
 - `internal void MoveObjectAt(Vector3Int newPosition, Vector3Int oldPosition)` - aktualizacja pozycji obiektu na mapie.
 - `public bool CanPlaceObjectAt(Vector3Int gridPosition)` - metoda sprawdzająca, czy pozycja na mapie nie jest zajęta przez inną płytkę.
2. `Tile` - Klasa reprezentująca pojedynczą płytkę w grze.
 - `public bool CheckTileValidity(Vector3Int gridPosition)` - metoda do sprawdzenia, czy daną płytkę można postawić zgodnie z zasadami gry Rummikub.
3. `BoardDictionary` - Klasa do przechowywania informacji o pozycjach płytek na mapie. Głównie służy do walidowania zasad gry planszowej w `TurnController`.
4. `FirstTurnController` - Klasa, która waliduje zasadę pierwszej tury w grze Rummikub dla użytkownika i agenta.
 - `public bool FirstJokerNull()`, `public bool SecondJokerNull()` - metody do sprawdzania, czy w pierwszej turze został wykorzystany joker.
 - `public void Increment(int number, Vector3Int gridPosition)` - metoda do dodawania wartości liczbowej do sumy punktów przy dodawaniu płytki na mapę.

- `public void Decrease(int number, Vector3Int gridPosition)` - metoda odejmująca od sumy w przypadku usuwania płytki z mapy w pierwszej turze.
- `public void ChangePosition(Vector3Int gridPosition, Vector3Int previousPosition)` - metoda zapisująca zmianę pozycji jokerów wykorzystanych w pierwszej turze.
- `public bool CheckSequences()` - sprawdza czy położone płytki w pierwszej turze są czystymi sekwencjami.
- `public void AddPosition(Vector3Int position)` - metoda dodaje do listy dane `Tile`.
- `public void RemovePosition(Vector3Int position)` - metoda usuwająca dane z listy.
- `public void MovePosition(Vector3Int newPosition, Vector3Int previousPosition)` - metoda aktualizująca pozycję płytki.
- `public int GetJokerAmount(Vector3Int jokerPosition, Dictionary<Vector3Int, Tile> board)` - metoda licząca, jaką liczbę joker zastępuje w celu wyliczenia sumy pod koniec tury.
- `public bool CheckFirstTurnValidity(Dictionary<Vector3Int, Tile> board)` - Metoda licząca sumę punktów w pierwszej turze gry.

5.6 Moduł użytkownika

`Player` - Klasa użytkownika, odpowiada za przechowywanie informacji o płytkach na stojaku gracza.

- `public void AddTileToList(Tile tile)` - Dodanie nowej płytki do stojaka gracza.
- `public void SortByNumbers()` - metoda do sortowania płytek w liście po numerach.
- `public void SortByColors()` - metoda sortowania płytek w liście gracza po kolorach.
- `public int FinalScore()` - metoda wyliczająca punktację użytkownika pod koniec gry.

5.7 Moduł gracza komputerowego

`ComputerPlayer` - Klasa gracza komputerowego z predefiniowanymi funkcjami.

- `public void SortByNumbers()`, `public void SortByColors()` - metody do sortowania płytek na stojaku.

- `private int CountJoker(List<Tile> playerHand)` - metoda do wyliczenia, ile gracz ma jokerów na stojaku.
- `public List<List<Tile>> FindSequentialColorSets(ref List<Tile> tiles)` - metoda do analizy płytek i tworzenia serii, które można postawić na mapie.
- `public List<List<Tile>> FindSameNumberDifferentColorSets( ref List<Tile> tiles)` - metoda do analizy płytek i tworzenia grup, które można postawić na mapie.
- `private bool IsSequenceValid(List<Tile> sequence)` - sprawdza, czy utworzone sekwencje mają co najmniej płytki w pojedynczej sekwencji.
- `private bool IsTotalSumValid(List<List<Tile>> sequences)` - sprawdza, czy znalezione sekwencje w pierwszej turze mają łączną wartość 30 punktów.
- `private bool IsTotalSumValid(List<List<Tile>> sequences)` - sprawdzenie czy dodanie jokera do sekwencji jest zgodne z zasadami. Dodanie jokera na końcu serii, która kończy się numerem 13 jest wbrew zasadom.
- `private List<Vector3Int> FreeSpaceToPut(int tileAmount)` - metoda, która analizuje, gdzie na mapie znajduje się wolna pozycja do położenia płytek.
- `private bool CheckPlacementValidity(Vector3Int gridPosition, int selectedObjectIndex)` - metoda do analizy, czy położenie płytki na danym polu jest zgodne z zasadami gry.
- `void PutTile(Vector3Int gridPosition, Tile tile)` - metoda do położenia pojedynczej płytki na mapie.
- `void ExtendSequence(ref Dictionary<Vector3Int, Tile> board, ref List<Tile> handTiles)` - metoda do położenia całej sekwencji płytek.
- `List<Vector3Int> FindSequencesNeighbours(Dictionary<Vector3Int, Tile> board)` - metoda znajduje pozycje na mapie, do których można dołożyć płytki.
- `List<Vector3Int> MoveSequence(ref Dictionary<Vector3Int, Tile> board, List<Vector3Int> freePositions)` - metoda do przesuwania całej sekwencji na mapie.
- `public int FinalScore()` - metoda do wyliczenia ostatecznego wyniku gracza pod koniec gry.

5.8 Moduł uczenia maszynowego

PlayerAI - Klasa dziedzicząca metody po interfejsie Agent, dostarczonej przez bibliotekę ML-Agents. Implementuje akcje gracza.

- `public override void CollectObservations(VectorSensor sensor)` - metoda odziedziczona od interfejsu, służy do wykonywania obserwacji zmiennych gry.
- `public override void OnActionReceived(ActionBuffers actions)` - metoda odziedziczona od interfejsu, implementuje funkcje akcjom wykonywanym przez agenta.
- `public override void WriteDiscreteActionMask(IDiscreteActionMask actionMask)` - metoda odziedziczona od interfejsu, ogranicza wybrane akcje agentowi, w celu uniknięcia błędów.
- `void PutTile(Vector3Int gridPosition, Tile tile)` - metoda do położenia płytki przez gracza
- `void PutTileAction(int x, int z, int indeks)` - metoda do walidacji akcji położenia płytki. Metoda daje nagrody w zależności od poprawności akcji agenta.
- `private bool CheckPlacementValidity(Vector3Int gridPosition, int selectedObjectIndex, int index)` - metoda sprawdzająca, czy można na daną pozycję postawić płytkę.
- `void moveTile(Vector3Int newGridPosition, Vector3Int oldGridPosition)` - funkcja do przesunięcia płytki na inną pozycję.
- `void MoveTileAction(int oldX, int oldZ, int newX, int newZ)` - funkcja walidująca poprawność wybranej akcji agenta. Nadaje nagrody bądź kary w zależności od poprawności wykonanej akcji.
- `void removeTile(Vector3Int gridPosition)` - funkcja do położenia płytki na mapie
- `void RemoveTileAction(int x, int z)` - Funkcja walidująca akcje usunięcia płytki z mapy. Metoda nadaje karę bądź nagrodę w zależności od poprawności wykonanego ruchu.
- `void EndOfTime()` - metoda cofająca modyfikacje, jakie wykonał agent, pod koniec czasu. Daje również agentowi karę za to.
- `public int FinalScore()`
- `void TakeTileAction()` - akcja pobrania płytki do stojaku.
- `void UndoAction()` - akcja anulowania wszystkich modyfikacji wykonanych na mapie.
- `void TakeTileAction()` - akcja zakończenia tury przez agenta.

5.9 Struktury danych

Aplikacja musi być w środowisku 2.5D, co wymaga zaimplementowania obiektów w przestrzeni 3D.

Stojak gracza jest obiektem 2D, każda płytką, jaką gracz posiada, jest obiektem typu `photo` z dynamicznie zmiennym tekstem. W pamięci, stojak jest przechowywany jako lista obiektów klasy `Tile`.

Jednak sama mapa, jak i obiekty na mapie, są zaimplementowane w przestrzeni 3D. W pamięci systemu, mapa jest reprezentowana jako słownik płytek `Dictionary<Vector3Int, Tile>`, gdzie każda lokalna pozycja na mapie jest zapisana jako klucz. Mapa, widziana przez użytkownika, opiera się na funkcjach siatki, jest to wbudowana funkcjonalność środowiska Unity. Siatka składa się z wierszy i kolumn, które pozwalają na jednoznaczne określenie pozycji obiektów na mapie. System siatki posiada wiele różnych zaimplementowanych funkcji dostępne na stronie [7], najważniejsze metody wykorzystane w projekcie:

- `CellToWorld(Vector3Int)` - Wartością wejściową funkcji jest Wektor 3D z wartościami całkowitymi, zwraca natomiast Wektor 3D wartości zmiennoprzecinkowych. Pozwala na wyciągnięcie pozycji obiektu w przestrzeni globalnej w czasie interakcji z nimi.
- `WorldToCell(Vector3)` - funkcja interpretująca pozycje świata obiektu do pozycji lokalnej siatki. Zmienną wejściową do funkcji jest Wektor 3D z wartościami zmiennoprzecinkowymi. Funkcja zwraca Wektor 3D liczb całkowitych

Wykorzystanie tych funkcji pozwala na łatwiejsze przechowywanie danych i pozycji na planszy.

5.10 System przemieszczania

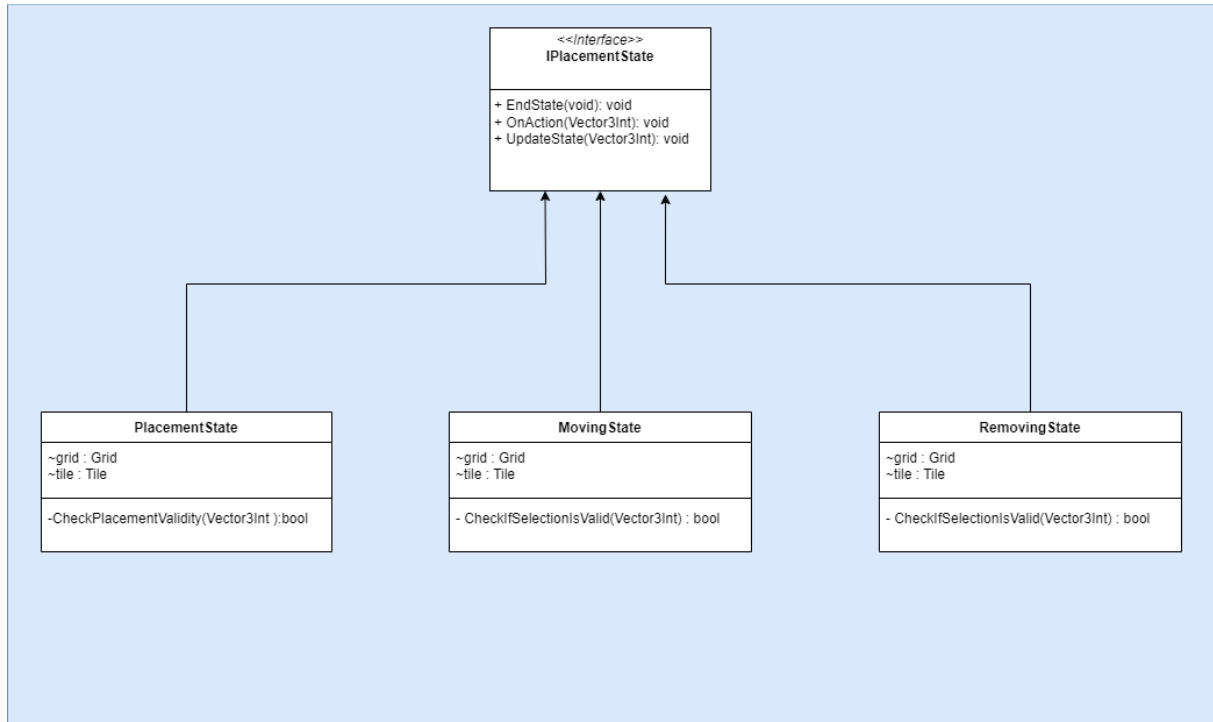
System przemieszczania w aplikacji opiera się na własnym interfejsie `IPlacementState`, który definiuje wspólny zestaw funkcji umożliwiających zarządzanie obiektami na mapie siatki (grid). Interfejs został zaimplementowany przez trzy klasy: `PlacementState`, `RemovingState` oraz `MovingState`, które różnią się sposobem inicjowania i realizacji funkcji.

```

1      public interface IPlacementState
2      {
3          void EndState();
4          void UpdateState(Vector3Int gridPosition);
5          void OnAction(Vector3Int gridPosition);
6      }

```

Każda interakcja użytkownika z obiektem tworzy nowy obiekt `IPlacementState`. Gdy gracz zakończy wybraną akcję, zaimplementowany interfejs zostaje usunięty.



Rysunek 5.1: Uproszczony diagram IPlacement

5.11 Biblioteka ML-Agents

Unity ML-Agents Toolkit to biblioteka służąca do implementowania i trenowania agentów z algorytmami uczenia maszynowego. Biblioteka pozwala na wykorzystanie metod takich jak uczenie ze wzmocnieniem, uczenie nadzorowane oraz uczenie w grupie. Biblioteka ML-Agents została wybrana do implementacji przeciwnika komputerowego w grze Rummikub, ponieważ:

- Umożliwia łatwe tworzenie środowiska treningowego w Unity.
- Pozwala dostosowywać metody modelu bezpośrednio do mechanik gry.
- Daje możliwość modyfikacji zmiennych modelu.

Kluczowymi komponentami ML-Agents są obserwacje, akcje oraz nagrody.

1. Obserwacje dają agentowi potrzebne informacje, aby zaplanować swoje kolejne ruchy. Ilość obserwacji jest ustalana przy implementacji statycznie, nie ma możliwości zmieniać ilości obserwacji w trakcie rozgrywki. Ich ilość jest podawana w obiekcie agenta w środowisku Unity.

2. Akcje są funkcjami, jakie podejmuje na bazie obserwacji oraz nagród, jakie otrzymuje. Agent podejmuje akcje, dzielą się one na dwa typy:
 - akcje ciągłe - model wyznacza akcje z podanego przedziału liczbowego w wartościach zmiennoprzecinkowych, zazwyczaj od -1 do 1. Takie akcje są najlepsze przy funkcjach ruchu czy rotacji.
 - akcje dyskretne - akcje modelu z podanego przedziału liczbowego w wartościach całkowitych. Najlepsze do konkretnych decyzji, jak wybieranie indeksów płytek w liście.
3. Otrzymywanie nagród i kar za konkretne działania dają agentowi informacje, jakie ruchy powinien wykonywać a jakie nie.

Decyzje agenta są interpretowane jako wartości liczbowe, przy implementacji należy przypisać konkretne wartości funkcji. W zależności od decyzji agenta, należy mu przypisać nagrodę bądź karę. Z czasem agent uczy się, jakie decyzje są najbardziej opłacalne i częściej je wykonuje.

Zmienne modelu mogą być modyfikowane w plikach typu `.yaml`. Na stronie [10] jest to dokładniej opisane.

Listing 5.1: Domyślny format pliku `.yaml`.

```

1 behaviors:
2   behaviors_name:
3     trainer_type: ppo
4     hyperparameters:
5       batch_size: 64
6       buffer_size: 2048
7       learning_rate: 3.0e-4
8       beta: 5.0e-4
9       epsilon: 0.2
10      lambda: 0.95
11      num_epoch: 3
12      learning_rate_schedule: linear
13      beta_schedule: constant
14      epsilon_schedule: linear
15    network_settings:
16      normalize: true
17      hidden_units: 128
18      num_layers: 2
19    reward_signals:
20      extrinsic:

```

```
21         gamma: 0.99
22         strength: 1.0
23     max_steps: 500000
24     time_horizon: 64
25     summary_freq: 10000
```

Kod Listing 5.1 prezentuje domyślną konfigurację. Modyfikacja środowiska pozwala na dostosowanie modelu do aplikacji, w której jest implementowane.

Każdy parametr odpowiada innej funkcjonalności modelu:

- **behaviors** - jest główną sekcją definiującą zachowanie agenta. Zawiera wszystkie szczegóły dotyczące jego treningu.
- **behaviors_name** - nazwa konkretnego zachowania, musi być identyczna do zaimplementowanej nazwy skryptu agenta podpiętego do obiektu w Unity.
- **trainer_type** - Typ algorytmu używanego do treningu. W przykładzie zawarty jest algorytm ppo (Proximal Policy Optimization)[6], który korzysta z warstw sieci neuronowej. Dzięki zastosowaniu sieci neuronowych algorytm jest w stanie skutecznie działać w środowiskach o dużej złożoności i wysokiej wymiarowości danych.
- **hyperparameters** - przedział zmiennych algorytmu:
 1. **batch_size** - liczba danych przetwarzanych w jednej iteracji.
 2. **buffer_size** - liczba przechowywanych danych w pamięci bufora przed aktualizacją sieci.
 3. **learning_rate** - określone tempo aktualizacji wag modelu podczas każdej iteracji.
 4. **beta** - współczynnik entropii, im wyższa wartość, tym model jest bardziej skłonny do eksploracji nowych strategii podczas nauki.
 5. **epsilon** - wyznacza maksymalną wartość, z jaką może zmieniać się polityka (policy) w jednej aktualizacji.
 6. **lambda** - współczynnik wygaśnięcia, do balansowania między dokładnością a stabilnością wyceny nagrody.
 7. **num_epoch** - liczba iteracji treningowych, podczas których model aktualizuje swoje wagi dla każdej partii danych.
 8. **learning_rate_schedule**, **beta_schedule**, **epsilon_schedule** - określają jak wartości zmiennych **learning_rate**, **beta** oraz **epsilon** zmieniają się w czasie treningu.
- **network_settings** - definiuje strukturę sieci neuronalnej, której używa agent:

1. `normalize` - definiuje, czy dane wejściowe mają zostać znormalizowane czy nie.
 2. `hidden_units` - liczba jednostek w każdej warstwie ukrytej sieci neuronalnej.
 3. `num_layers` - ilość warstw sieci neuronalnej.
- **`reward_signals`** - definiuje sygnał nagrody dla agenta. Tutaj jest użyty typ `extrinsic`, czyli nagroda bazująca na środowisku:
 1. `gamma` - współczynnik wygaśnięcia, który określa w jakim stopniu agent bierze pod uwagę przyszłe nagrody.
 2. `strength` - skala wpływu sygnału na trening.
 - **`max_steps`** - Maksymalna liczba kroków treningowych, definiowanych przy implementacji, jakie może wykonać agent. W tym projekcie, jest to pojedyncza gra. Po osiągnięciu tej wartości, model nie może być dalej uczony.
 - **`time_horizon`** - maksymalny czas, po jakim agent kończy zbieranie danych z obserwacji do bufora.
 - **`summary_freq`** - określa, co ile zostają wygenerowane dane podsumowania treningu.

Parametry środowiskowe mogą się różnić w zależności od wybranego `trainer_type`. Przy grze Rummikub ważne jest, aby dostosować parametry do dużej ilości obserwacji oraz akcji.

5.12 Klasa PlayerIA

Klasa `PlayerIA` implementuje metody dla agenta do wykonywania konkretnych funkcjonalności i interakcji w środowisku. Klasa dziedziczy po interfejsie `Agent`, która jest dostarczana przez bibliotekę `ML-Agents`. Najważniejsze funkcje, jakie klasa odziedziczyła po interfejsie to :

- `CollectObservations` - funkcja, służąca do zbierania obserwacji przez agenta
- `OnActionReceived` - funkcja do przechwytywania akcji modelu i implementowania im funkcjonalności.
- `AddReward` - przesyła nagrody bądź kary do modelu.

Akcje oraz obserwacje w systemie `ML-Agents` są definiowane w edytorze Unity, a nie bezpośrednio w kodzie. Oznacza to, że ich liczba i zakres muszą być statycznie ustalone przed rozpoczęciem treningu. Nie można ich dynamicznie zmieniać w czasie działania gry. Aby uniknąć problemów z ograniczeniami, należy ustawić maksymalne możliwe wartości dla

liczby akcji i obserwacji. Każda obserwacja w ML-Agents dotyczy pojedynczych zmiennych. Jeśli chcemy, aby agent mógł obserwować obiekt, który ma wiele właściwości (np. klasę `Tile`), liczba obserwacji musi odpowiadać liczbie wszystkich zmiennych w tej klasie. W klasie `PlayerIA` pod obserwacje należało poddać:

- Ilość płytek na stojaku.
- Płytki, jakie agent ma w stojaku.
- Czy to jego pierwsza tura.
- Aktualny czas tury.
- Płytki oraz puste miejsca na mapie.

Wszystkie obserwacje mają na celu pokazanie agentowi, jak obiekty się zmieniają gdy dostaje konkretne nagrody.

Akcje modelu są liczbami, którym przypisane są konkretne funkcje. Akcje mają swoje przedziały, które są mu przypisane. Agent ma przypisane sześć akcji w różnych rozmiarach:

- akcja 0 - Wybór wartości jako pozycji x na mapie.
- akcja 1 - Wybór wartości jako pozycji z na mapie.
- akcja 2 - Wybór konkretnej akcji jak kładzenie płytek czy usuwanie płytki.
- akcja 3 - akcja do wybierania indeksu płytek ze stojaka agenta w celu położenia płytki.
- akcja 4 - Wybór wartości x jako pozycji na mapie, wykorzystany przy przesuwaniu płytki.
- akcja 5 - Wybór wartości z jako pozycji na mapie, wykorzystany przy przesuwaniu płytki.

Akcje są ustawiane statycznie, nie można ich ustawiać dynamicznie. Ilość płytek, na stojaku gracza, jest dynamiczna. Na jedną rozgrywkę z czterema graczami, model w najgorszym przypadku może maksymalnie dobrać 64 płytki, co wymusza takie ustawienie. Właśnie dlatego interfejs Agent daje możliwość zaimplementować funkcję `WriteDiscreteActionMask`, która pozwala na maskowanie nadmiarowych akcji. Dzięki czemu agent nie podejmie decyzji interakcji z płytką, której nie ma w liście.

Rozdział 6

Weryfikacja i walidacja

Zakres pracy projektu obejmował spełnienie podanych kryteriów:

- Gra planszowa.
- W przestrzeni 2.5D.
- W środowisku Unity.
- Uczenie maszynowe komputerowych graczy.

Wszystkie powyższe warunki zostały pomyślnie spełnione w trakcie realizacji projektu.

6.1 Testowanie

Zostały wykonane testy manualne w celu sprawdzania poprawności działania głównych funkcjonalności gry. Testom podlegały również modele uczenia maszynowego, były implementowane różne nagrody oraz zmieniane parametry w pliku `.yaml`, aby znaleźć najlepszą kombinację, dzięki której model się najlepiej uczy.

6.1.1 Testy manualne

1. Kładzenie płytek na mapę - celem było sprawdzenie, czy płytki tworzą się poprawnie i dane, takie jak liczba, kolor oraz pozycja, zostaną poprawnie zapisane do słownika. Przebieg:

- Rozmieszczenie płytki na mapę.
- Próba położenia płytki na zajętej już pozycji.
- Ocena poprawności wyniku na bazie informacji zwrotnych w grze.

Oczekiwany wynik: wszystkie obiekty tworzone na mapie poprawnie zapisują się słownika.

2. Tworzenie grup i serii - celem była weryfikacja, czy użytkownik i gracze komputerowi potrafią tworzyć sekwencje na mapie zgodnie z zasadami gry. Przebieg:

- Ręczne rozmieszczanie płytek na mapę.
- Próba utworzenia sekwencji lub grupy zgodnie z zasadami.
- Próba utworzenia sekwencji bądź grupy wbrew zasadom.
- Ocena wyników w oparciu o informacje zwrotne.

Oczekiwany wynik: system poprawnie rozpoznaje prawidłowe i nieprawidłowe serie oraz grupy.

3. Obsługa jokerów - Sprawdzenie, czy joker poprawnie zastępuje daną płytkę na mapie. Przebieg:

- Dodanie jokera do serii.
- Dołożenie kolejnej płytki do serii obok jokera.
- Ocena wyników.

Oczekiwany wynik: system poprawnie wykrywa jokery jako uniwersalne płytki.

4. Pierwsza tura z minimalną sumą punktów 30 - Upewnienie się, że pierwsza tura każdego gracza zakończy się czystymi sekwencjami z minimalnymi sumami wartości płytek wynoszącej 30 punktów. Przebieg:

- Wyłożenie sekwencji o sumie punktów mniejszej niż 30
- Próba zakończenia tury.
- Wyłożenie sekwencji o sumie punktów równej bądź większej niż 30.
- Próba zakończenia tury.
- Wyłożenie sekwencji z jokerami o sumie punktów mniejszej niż 30.
- Próba zakończenia tury.
- Wyłożenie sekwencji z jokerami o sumie punktów równej bądź większej niż 30.

Oczekiwany wynik: system poprawnie wylicza punkty położone płytki w pierwszej turze

5. Interfejs użytkownika - Sprawdzenie intuicyjności, funkcjonalności przycisków oraz sprawdzanie komunikatów wyświetlanych użytkownikowi. Przebieg:

- Interakcja z płytkami na stojaku użytkownika.
- Interakcja z przyciskami na ekranie

- Testowanie ruchów na mapie z poprawnym rozpoznawaniem błędnych zagrań przez system.
- Ocena informacji zwrotnych wyświetlanych graczowi (błędy, sukcesy, komunikaty).

Oczekiwany wynik: interfejs jest klarowny dla użytkownika i wszystkie funkcje działają zgodnie z założeniami.

6. System tur - sprawdzenie, czy funkcjonalności dotyczące zakończenia tury oraz interakcje poza turą działają poprawnie. Przebieg:

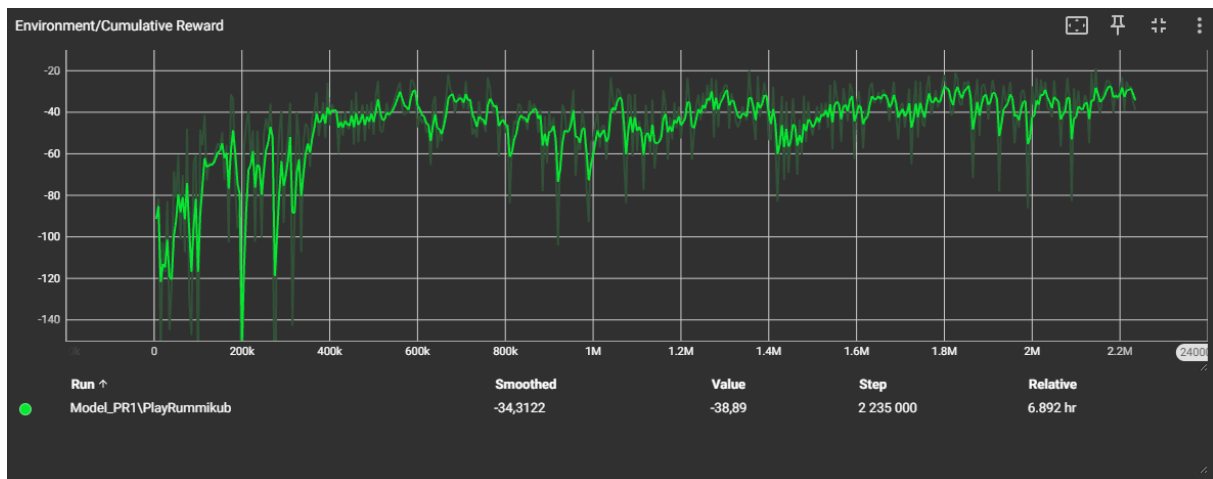
- Wyłożenie za krótkiej sekwencji.
- Próba zakończenia tury.
- Wyłożenie poprawnej sekwencji.
- Próba zakończenia tury.
- Próba interakcji z mapą poza turą.

Oczekiwany wynik: System poprawnie zablokuje zakończenie tury, gdy co najmniej jedna sekwencja na mapie jest niepoprawna. Również system blokuje możliwość wykonywania interakcji z mapą, gdy nie ma tury danego użytkownika.

6.1.2 Modele

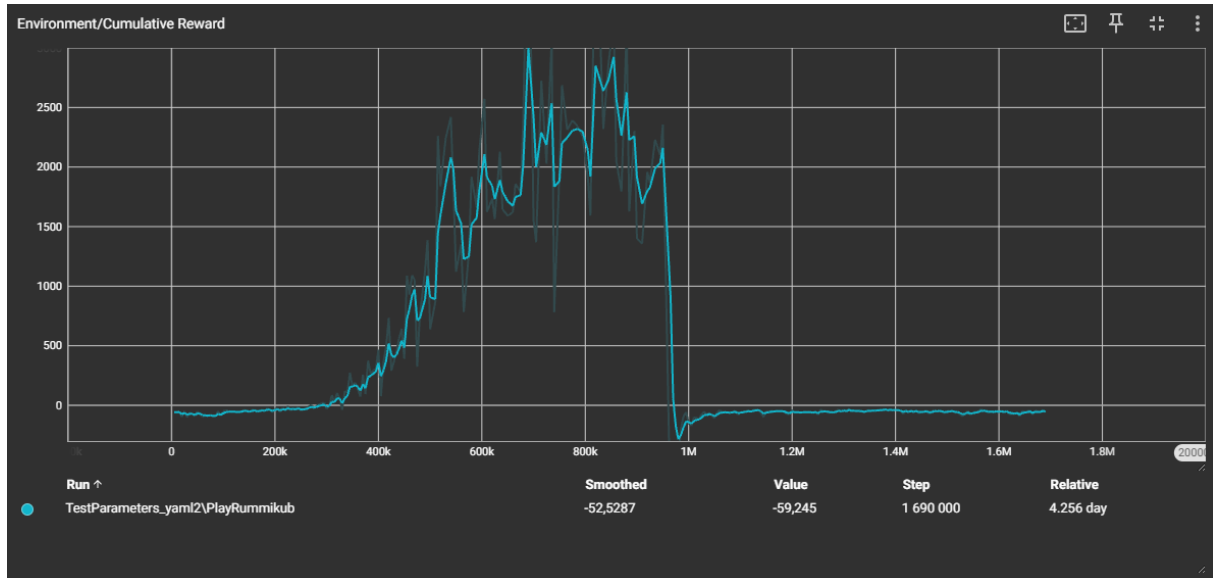
W trakcie trwania projektu, zostały podjęte próby wyuczenia kilku modeli z różnymi parametrami i nagrodami. Wyniki:

- Model nr 1 : wyniki modelu, widoczne na rysunku 6.1, wskazują na brak poprawy wyniku przy dłuższym czasie nauki.



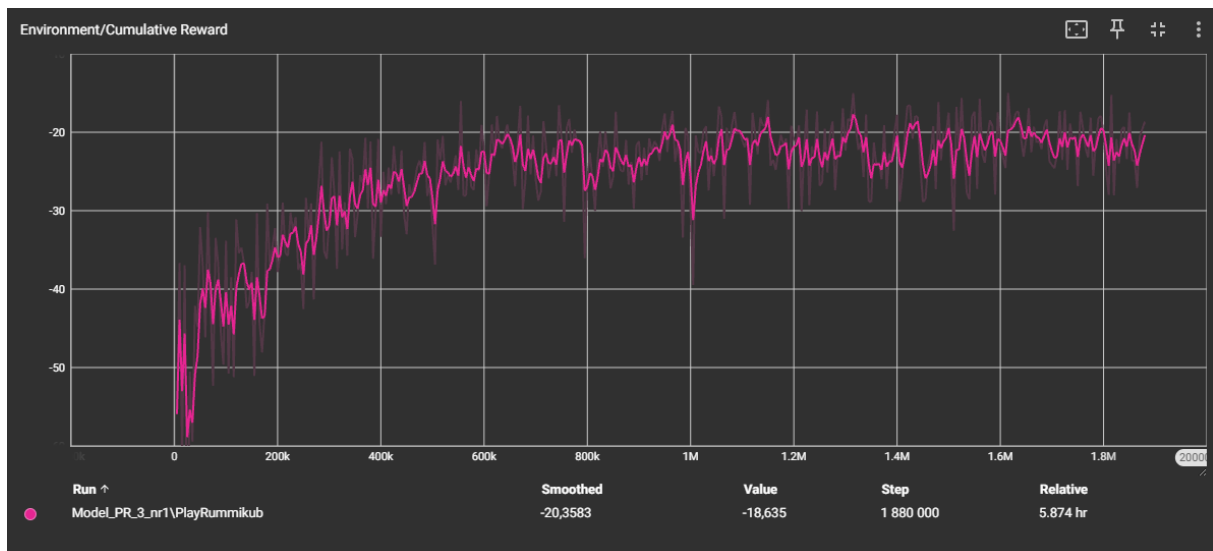
Rysunek 6.1: Wykres przydzielonych nagród modelowi nr 1

- Model nr 2 : Została podjęta próba zwiększenia nagrody za samo kładzenie płytek. Po pewnym czasie przyznawane nagrody zostały zmienione, aby model nauczył się tworzyć sekwencje. Jednak nie przyniosło to oczekiwanych rezultatów, co widać na rysunku 6.2.



Rysunek 6.2: Wykres przydzielonych nagród modelowi nr 2

- Model nr 3 : Zostały zmienione parametry konfiguracyjne modeli, jednak rezultat jest podobny do wyników modelu nr 1, co widać na rysunku 6.3.



Rysunek 6.3: Wykres przydzielonych nagród modelowi nr 3

Jak wynika z przedstawionych wykresów na rysunkach 6.1, 6.2, 6.3 modele mają trudności z adaptacją do środowiska gry. Gra planszowa Rummikub jest skomplikowaną grą strategiczną, co stanowi wyzwanie dla modeli, aby nauczyć się zasad gry. Proces nauki

przebiega w trybie prób i błędów, co powoduje, że zamiast optymalizować swoje działania w kierunku maksymalizacji nagród, modele skupiają się na minimalizowaniu kar. Taka strategia nie pozwala na skuteczne opanowanie zasad gry i osiągnięcie zamierzonych celów treningowych.

6.2 Wykryte błędy

Podczas implementacji projektu pojawiały się błędy, które wpływały na działanie aplikacji. Poniżej przedstawiono najistotniejsze z nich:

1. Błędne zabezpieczenie użyteczności jokerów :

Jokery mogą zastępować każdą płytkę, są jednak pewne ograniczenia względem sekwencji i grup. Serie mogą występować w maksymalnie ciągach od 1 do 13. Grupy mogą występować w ilości maksymalnie 4 na ciąg. Jokery były niepoprawnie zaimplementowane, przez co można było dołożyć jokera do już zapełnionej grupy, bądź można było dodać jokera na koniec bądź początek pełnej serii, gdzie joker zastępował wartości 0 bądź 14, czego nie powinno być w grze.

2. Błędnie zainicjowana funkcja tworzenia sekwencji:

Gracze komputerowi z predefiniowanymi ruchami mają gotowe funkcje, które wykonują w trakcie swojej tury. Błąd był do zauważenia, gdy gracz miał swoją pierwszą turę. Wtedy należy wyłożyć sumę punktów równą bądź większą od 30 i musi być to wykonane czystymi sekwencjami, może być to kilka sekwencji. Jednak gracze brali pod uwagę tylko jedną sekwencję na ruch, mimo że ogólną sumę by mieli, przez co mogli się nie wyłożyć przez długi czas.

3. Przypisanie akcji agentowi:

Akcje agenta opierają się na wyborze wartości w zaimplementowanym przedziale, same liczby nie mają znaczenia, dopóki nie zostaną im przypisane funkcje. W kodzie, każdy przedział jest zapisany jako obiekt w tablicy:

```
1  int akcjaAgenta = actions.DiscreteActions[2];
```

Pod akcje agenta został podpisany błędny indeks, co doprowadziło do błędnych wyników nauki.

4. Niezabezpieczone maskowanie akcji:

Jeden przedział ma wartości od 0 do 63, dotyczy on wyboru płytek po indeksach. Jednak agent nie może wybrać liczby indeksu 14, gdy ma przykładowo 10 płytek na stojaku. W tym celu pojawia się maskowanie akcji, ogranicza to możliwość wyboru

modelowi na dany moment, względem ilości płytek. Jednak gracz mając akcje, zawsze musi w ramach niej wybrać liczbę, a gdy gracz miał pusty stojak, to funkcja zamaskowała każdą wartość, przez co gracz nie mógł wykonać żadnej akcji, co nie powinno się wydarzyć. Obecnie, gdy agent ma pusty stojak, to funkcja maskująca nie działa, oraz wybrane akcje kładzenia płytek nie działają.

Rozdział 7

Podsumowanie i wnioski

Wykonanie projektu Gra planszowa 2.5D z uczeniem maszynowym komputerowych graczy miało na celu zaimplementowanie gry planszowej 2.5D w środowisku Unity z maszynowo uczącymi się komputerowymi graczami. Proces realizacji projektu obejmował implementację mechanizmów gry oraz gracza komputerowego z algorytmami uczenia maszynowego. W trakcie projektu napotkano wiele wyzwań związanych z implementacją zasad gry, poprawnie działających funkcji graczy komputerowych oraz nauką agenta.

Efektem końcowym jest prototyp gry, który spełnia założenia funkcjonalne, takie jak możliwość rozgrywki między użytkownikiem a graczami komputerowymi, dynamiczne zarządzanie planszą oraz, zgodne z zasadami gry planszowej, mechaniki jokerów i pierwszej tury. Projekt pozwolił również na przeprowadzenie eksperymentów z uczeniem maszynowym, które ujawniły trudności w nauce zasad Rummikub przez agentów AI, szczególnie w kontekście złożoności strategii wymaganej do optymalnego działania.

7.1 Napotkane problemy

W trakcie implementacji projektu pojawiło się kilka istotnych problemów, które wpłynęły na tempo pracy.

- Złożoność zasad gry

Zasady gry planszowej Rummikub są bardzo złożone, biorąc pod uwagę jokery, ograniczenia wielkości grup do 4 i serii do 13 płytek oraz zasadę pierwszej tury z punktacją równą minimalnie 30 punktów. Przez to implementacja trwała dłużej niż zakładano.

- Nauka agentów

Rummikub przez swoją złożoność, jest bardzo trudną grą do nauki graczy komputerowych z algorytmami uczenia maszynowego. Dotychczasowe wyuczone modele nie zdołały nauczyć się zasady pierwszej tury, przez co zamiast eksplorować różne akcje

w celu optymalizacji ruchów i zyskiwaniu największych nagród, dążyły do uzyskania najmniejszej kary. Jest to problematyczne, ponieważ taki gracz nie będzie w stanie optymalnie brać udziału w rozgrywce.

7.2 Możliwy rozwój

Zaimplementowany projekt jest możliwy do rozwoju w celu dodania nowych funkcjonalności, które mogłyby uatrakcyjnić rozgrywkę. Możliwe kierunki rozwoju:

- Udoskonalenie funkcjonalności graczy komputerowych z predefiniowanymi ruchami

Ich obecne funkcje polegają na tworzeniu nowych sekwencji i dokładanie płytek do istniejących. Nowymi potencjalnymi funkcjonalnościami dla graczy to rozkładanie większych sekwencji na mniejsze i dokładanie do nich nowych płytek. Gracz też mógłby zastąpić jokera na mapie, jeśli ma płytkę którą joker zastępuje, a następnie wykorzystać jokera do stworzenia nowej sekwencji.

- Dodanie nowych trybów gry

W projekcie gry, został uwzględniony podstawowy tryb gry planszowej Rummikub z 4 osobami oraz 106 płytkami. Warto jednak zaznaczyć, że producent gry opracował także rozszerzoną wersję przeznaczoną dla 5–6 graczy, w której do dyspozycji uczestników pozostaje aż 160 płytek. Można by zaimplementować ją do tego projektu.

- Gra online

Projekt uwzględniał implementację graczy komputerowych z algorytmami uczenia maszynowego. Aby skupić się na tym aspekcie, w projekcie została pominięta implementacja trybu online, gdzie kilku użytkowników rywalizowałoby.

- Nowi inteligentni gracze

W projekcie został zaimplementowany gracz komputerowy z algorytmem uczenia maszynowego z wykorzystaniem **ML-Agents**. W projekcie można by zaimplementować nowych graczy, którzy wykorzystywaliby inne algorytmy sztucznej inteligencji.

Bibliografia

- [1] Stockfish Developers. *Stockfish - Open Source Chess Engine*. 2024. URL: <https://github.com/official-stockfish/Stockfish> (term. wiz. 22.12.2024).
- [2] Python Software Foundation. *Beginners Guide to Python: Download*. 2024. URL: <https://wiki.python.org/moin/BeginnersGuide/Download> (term. wiz. 05.11.2024).
- [3] Python Software Foundation. *The Python Programming Language*. 2024. URL: <https://www.python.org/doc/essays/blurbs/> (term. wiz. 22.12.2024).
- [4] Microsoft. *Przewodnik po języku C#*. 2024. URL: <https://learn.microsoft.com/pl-pl/dotnet/csharp/> (term. wiz. 22.12.2024).
- [5] Rummikub. *Zasady gry Rummikub*. 2019. URL: <https://rummikub.com/wp-content/uploads/2019/12/2600-English-1.pdf> (term. wiz. 15.12.2024).
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford i Oleg Klimov. „Proximal policy optimization algorithms”. W: *arXiv preprint arXiv:1707.06347* (2017).
- [7] Unity Technologies. *Grid - Unity Scripting API*. 2024. URL: <https://docs.unity3d.com/6000.0/Documentation/ScriptReference/Grid.html> (term. wiz. 02.01.2025).
- [8] Unity Technologies. *Installing Unity*. 2016. URL: <https://docs.unity3d.com/550/Documentation/Manual/InstallingUnity.html> (term. wiz. 11.12.2024).
- [9] Unity Technologies. *ML-Agents Toolkit*. 2024. URL: <https://github.com/Unity-Technologies/ml-agents> (term. wiz. 22.12.2024).
- [10] Unity Technologies. *ML-Agents Toolkit: Creating New Learning Environments*. 2024. URL: <https://github.com/Unity-Technologies/ml-agents/blob/develop/docs/Learning-Environment-Create-New.md> (term. wiz. 15.12.2024).
- [11] Unity Technologies. *ML-Agents Toolkit: Installation Guide*. 2024. URL: <https://github.com/Unity-Technologies/ml-agents/blob/develop/docs/Installation.md> (term. wiz. 11.12.2024).
- [12] Unity Technologies. *Unity - Game Development Platform*. 2024. URL: <https://unity.com/games> (term. wiz. 22.12.2024).

- [13] Unity Technologies. *Unity System Requirements*. 2023. URL: <https://docs.unity3d.com/2022.3/Documentation/Manual/system-requirements.html> (term. wiz. 10.12.2024).

Dodatki

Spis skrótów i symboli

AI, ai - Artificial Intelligence (Sztuczna Inteligencja)

ML – Machine Learning (Uczenie Maszynowe)

Unity – silnik do tworzenia gier

ML-Agents – Narzędzie w Unity do uczenia maszynowego

Joker – specjalna płytką w Rummikub, zastępująca dowolny numer.

Źródła

Lista dodatkowych plików, uzupełniających tekst pracy

Spis rysunków

2.1	Zrzut ekranu gry Business Tour	4
2.2	Zrzut ekranu gry Go na stronie online-go.com	4
2.3	Zrzut ekranu gry Catan	5
2.4	Zrzut ekranu rozgrywki szachów ze strony chess.com	5
2.5	Zrzut ekranu gry Rummikub online	6
3.1	Diagram przypadków użycia	10
4.1	Menu gry	15
4.2	Przykładowy ekran początku gry	16
4.3	Aktualna tura: Gracz X	16
4.4	widok listy graczy	17
4.5	widok stojaku z płytkami	17
4.6	przyciski poza turą	18
4.7	przyciski przed wykonaniem ruchu	18
4.8	przyciski po wykonaniu ruchu	19
4.9	komunikat o źle zakończonej mapie	20
4.10	komunikat o źle zakończonej pierwszej turze	20
4.11	komunikat braku wyłożonej płytki	20
4.12	komunikat przy niedozwolonym usuwaniu płytki	20
4.13	ekran końca gry	21
4.14	ekran końca gry przy pustym banku	21
4.15	jokery	22
4.16	Ekran zatrzymania gry	22
4.17	Kładzenie płytki na mapę	23
4.18	Złe położenie płytki	24
4.19	wygląd usuwania płytki	24
4.20	Włączanie nauczania w wierszu poleceń	25
4.21	Parametry modelu w wierszu poleceń	26
4.22	Aktualizacje modelu w wierszu poleceń	27
4.23	Uruchomienie strony z wynikami	27

4.24	Strona z wynikami	27
5.1	Uproszczony diagram IPlacement	40
6.1	Wykres przydzielonych nagród modelowi nr 1	47
6.2	Wykres przydzielonych nagród modelowi nr 2	48
6.3	Wykres przydzielonych nagród modelowi nr 3	48

Spis tabel